OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

CHAPTER 17

PROGRAMMING, DATA STRUCTURES & OPERATING SYSTEMS

SUGGESTED TIME: 4 PERIODS

IN THIS CHAPTER

This chapter consolidates Advanced Level Computer Science by linking three pillars: C programming, data structures, and operating systems. Students design and code a data indexing and search module in C, then build a CPU and memory allocation simulator that reproduces the core decisions of a real operating system.

- By the end of this chapter I can declare and manipulate fundamental C types, pointers, arrays, strings and structures.
- By the end of this chapter I can implement and trace linear data structures (arrays, linked lists, stacks, queues) in C.
- By the end of this chapter I can implement and compare linear search and binary search, and analyse their complexity.
- By the end of this chapter I can build a simple index (key → record) to speed up data retrieval in a file-based application.
- By the end of this chapter I can explain the role and main functions of an operating system (process, memory, file and device management).
- By the end of this chapter I can describe process states and apply CPU scheduling algorithms (FCFS, SJF, Round Robin) by hand and in code.

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+

CONTENTS

1. C Programming Foundations for Data Manipulation
2. Data Structures, Searching and Indexing
3. Operating Systems and System Software
4. Hands-on Projects: Indexing Module and OS Simulator
5. Integration activity
6. Methods and worked examples
7. Exercises
8. Solutions to the exercises
9. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can declare and manipulate fundamental C types, pointers, arrays, strings and structures.
- By the end of this chapter I can implement and trace linear data structures (arrays, linked lists, stacks, queues) in C.
- By the end of this chapter I can implement and compare linear search and binary search, and analyse their complexity.
- By the end of this chapter I can build a simple index (key → record) to speed up data retrieval in a file-based application.
- By the end of this chapter I can explain the role and main functions of an operating system (process, memory, file and device management).
- By the end of this chapter I can describe process states and apply CPU scheduling algorithms (FCFS, SJF, Round Robin) by hand and in code.
- By the end of this chapter I can simulate contiguous memory allocation strategies (first-fit, best-fit, worst-fit).
- By the end of this chapter I can design, code, compile, test and document a complete modular C project suitable for the GCE practical examination.

PREREQUISITES

- Algorithmics from Lower Sixth: variables, control structures (if, while, for), subpro-

grams and pseudocode.

- Basic programming notions: compilation vs interpretation, source code, syntax and logic errors.
- Number systems and data representation (binary, hexadecimal, memory units: byte, KB, MB).
- Notion of software classification: system software vs application software (Form 5 / Lower Sixth).
- File handling basics: opening, reading and writing a text file.

1 C PROGRAMMING FOUNDATIONS FOR DATA MANIPULATION

Every evening, the registrar of a large secondary school in Yaoundé searches through thousands of student records to produce transcripts before the GCE registration deadline — a task that takes hours when data is stored carelessly, but seconds when it is properly indexed. Meanwhile, the school's single computer lab must run a browser, a C compiler and a printer queue at the same time without freezing. How does a program organise data in memory so that searching is fast? And how does the operating system fairly share the CPU and RAM between competing programs? This chapter answers both questions by combining C programming, data structures and operating system concepts into two hands-on projects. We begin with the tool that makes everything else possible: the **C language**.

1.1 The C Language in the GCE Context

Why C?

C is a **compiled language**: the whole source text is translated once into machine code by a compiler, producing an executable file that runs directly on the processor. This contrasts with interpreted languages, where each line is translated at run time. Compilation makes C programs fast — exactly what we need when searching thousands of student records — and gives the programmer direct control over memory, which is why operating systems themselves are written in C.

DEFINITION **C program**

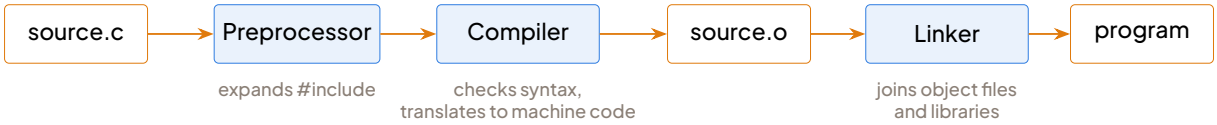
A **C program** is a text file (conventionally with the extension `.c`) containing instructions written in the C language, which must be translated by a compiler into an executable binary file before it can run.

Structure of a C program. Every C program follows the same skeleton:

```
#include <stdio.h>    // preprocessor directive: import standard input/output library
int main(void) {    // entry point of the program
    int age = 17;    // statement: declaration + initialisation
    printf("Age = %d\n", age);
    return 0;    // 0 signals success to the operating system
}
```

Three ingredients appear here. **Preprocessor directives** (lines starting with `#`) are handled before compilation: `#include` literally inserts the content of a header file. The function `main` is where execution always begins. **Statements** are the individual orders, each terminated by a semicolon.

The compilation chain. Turning `source.c` into a running program involves several stages, usually driven by a single command such as `gcc source.c -o program`:



The C compilation chain: preprocessing, compiling to an object file, then linking into an executable.

METHOD **Compiling and debugging a C program**

1. Compile with warnings enabled: `gcc -Wall source.c -o program`.
2. Read the first error message only; later errors are often consequences of the first.
3. Fix that error, recompile, repeat until compilation succeeds.
4. Run the program with test data whose expected result you know.
5. If the output is wrong, insert `printf` traces showing the values of key variables at strategic points to locate the logic error, then remove the traces.

1.2 Types, Input/Output and Operators

Fundamental types. C offers a small set of basic types:

Type	Meaning	Example
`int`	whole number	`int students = 45;`
`float`	real number (single precision)	`float mark = 13.5f;`
`double`	real number (double precision)	`double pi = 3.1415926535;`
`char`	single character	`char grade = 'A';`

A **constant** is declared with `const` (e.g. `const int MAX = 100;`) or with `#define MAX 100;` its value cannot change during execution.

Formatted input/output. `printf` writes to the screen and `scanf` reads from the keyboard; both use format specifiers: `%d` for `int`, `%f` for `float`/`double` (printing), `%lf` for `double` in `scanf`, `%c` for `char`, `%s` for strings.

- `int age;`
- `printf("Enter age: ");`
- `scanf("%d", &age);` // note the & : scanf needs the ADDRESS

- `printf("You are %d years old.\n", age);`

> **WARNING** **scanf without &**
>
> A very common mistake is writing `scanf("%d", age);` without the `&` operator on a non-pointer variable. `scanf` then writes the typed value to a meaningless memory location, which usually causes a segmentation fault (the program crashes). Always pass `&variable` for `int`, `float`, `double` and `char`.

Operators and casting. Besides `+ - * /`, the operator `%` gives the remainder of integer division (`17 % 5` is 2). Beware: dividing two `int` values discards the fractional part, so `7 / 2` is 3, not 3.5. A **cast** forces a conversion: `(float)7 / 2` gives 3.5.

> **WARNING** **= versus ==**
>
> In an `if` condition, `=` is assignment while `==` is comparison. Writing `if (mark = 10)` assigns 10 to `mark` and the condition is always true; the compiler may not even complain. Always write `if (mark == 10)`.

1.3 Control Structures in C

C provides the decision structures `if`/`else` and `switch`, and the loops `while`, `do-while` and `for`. Conditions use the comparison operators `==`, `!=`, `<`, `<=`, `>`, `>=` and the logical operators `&&` (AND), `||` (OR), `!` (NOT).

```
// Appreciation of a GCE mark out of 20
if (mark >= 16)      printf("Excellent");
else if (mark >= 10) printf("Pass");
else                 printf("Fail");
```

A counted loop that prints the multiplication table of 7:

```
int i;
for (i = 1; i <= 10; i++) {
    printf("7 x %d = %d\n", i, 7 * i);
}
```

Use `while` when the number of repetitions is unknown (e.g. reading until the user types -1), `do-while` when the body must run at least once (e.g. re-asking until input is valid), and `for` when a counter is natural. Always indent the body of each block: indentation is what makes nested structures readable and is expected in examination answers.

1.4 Functions and Modular Decomposition

A **function** is a named, reusable block of code that receives parameters and may return a value. Decomposing a problem into functions is the heart of good program design: the registrar's program might have `readRecords()`, `searchByMatricule()` and `printTranscript()`.

```
// Declaration (prototype), usually placed before main
float average(float a, float b);

// Definition
float average(float a, float b) {
    return (a + b) / 2.0f;
}
```

By default, C passes parameters **by value**: the function receives a copy, so it cannot modify the caller's variable. To let a function modify a variable, we pass its address — this is where pointers come in.

1.5 Arrays and Strings

An **array** stores a fixed number of elements of the same type in contiguous memory; indexing starts at 0.

```
float marks[5] = {12.5f, 9.0f, 15.0f, 11.0f, 14.5f};
marks[0] = 13.0f;   // first element
int grid[3][4];   // 2D array: 3 rows, 4 columns
```

A **string** in C is simply an array of `char` terminated by the special character `'\0'`. The library `string.h` provides essential tools: `strlen(s)` returns the length (without the terminator), `strcpy(dest, src)` copies a string, and `strcmp(a, b)` returns 0 when the strings are equal (negative/positive otherwise).

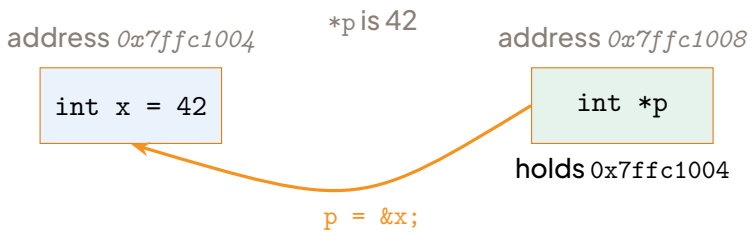
> **WARNING** **Strings are not compared with ==**
>
> `if (name == "Etoundi")` compares addresses, not contents. Always use `if (strcmp(name, "Etoundi") == 0)`.

1.6 Pointers and Dynamic Memory

Every variable lives somewhere in memory, at a numeric address. The operator `&` gives the address of a variable; the operator `*` accesses the value stored at an address.

DEFINITION **Pointer**

A **pointer** is a variable whose value is the memory address of another variable. If `p` points to `x`, then `*p` (dereferencing) is exactly `x`.



Boxes-and-arrows view of memory: the pointer `p` stores the address of `x`; dereferencing `*p` reaches the value 42.

Passing by reference. To swap two variables, the function must receive their addresses:

```
void swap(int *a, int *b) {
    int tmp = *a; *a = *b; *b = tmp;
}
// call: swap(&x, &y);
```

Pointer arithmetic. If `p` points to an element of an array, `p + 1` points to the next element (C automatically multiplies by the element size). This is why arrays and pointers are closely related: `marks[i]` is equivalent to `*(marks + i)`.

Dynamic allocation. When the number of records is unknown at compile time, we request memory at run time:

```
#include <stdlib.h>
int *tab = malloc(n * sizeof(int));    // n integers on the heap
if (tab == NULL) { /* allocation failed */ }
/* ... use tab ... */
free(tab);    // give the memory back
```

PROPERTY **Rule of dynamic memory**

Every block of memory obtained with `malloc` must be released exactly once with `free`. Forgetting `free` causes a memory leak (the program consumes more and more RAM); calling `free` twice, or using the pointer after `free`, causes undefined behaviour and

crashes.

1.7 Structures: Modelling a Record

A student is not a single number: it is a matricule, a name and several marks. C lets us group these fields under one name.

DEFINITION **Structure**

A **structure** (`struct`) groups several fields of different types under one name to represent a record. Each field is accessed with the dot operator (`record.field`), or with `->` through a pointer.

```
typedef struct {
    char matricule[12];
    char name[40];
    float marks[3];
} Student;

Student s;
strcpy(s.matricule, "GCE2025-001");
strcpy(s.name, "Ngo Bell");
s.marks[0] = 15.5f;
```

The `typedef` keyword gives the structure a short alias, so we write `Student s;` instead of `struct Student s;`. An array of structures then models the whole class: `Student class[60];` — and `class[i].marks[j]` is the mark *j* of student *i*. This is exactly the data model our indexing project will use.

1.8 Data Structures in C: Beyond Arrays

Arrays provide fast random access but have fixed size and costly insertion/deletion. For dynamic collections we use **linked structures** built from structures and pointers.

DEFINITION **Singly linked list**

A **singly linked list** is a sequence of nodes, each containing a data field and a pointer to the next node. The list is accessed through a pointer to the first node (`head`); the last node points to `NULL`.

- `typedef struct Node {`
- `    Student data;` // any payload
- `    struct Node *next;`
- `} Node;`
- `Node *head = NULL;`

Basic operations on a singly linked list:

METHOD **Insert a new node at the front of a list**

1. Allocate a new node: `Node *newNode = malloc(sizeof(Node));`
2. Check `newNode != NULL`.
3. Fill the data field: `newNode->data = studentRecord;`
4. Link it: `newNode->next = head;`
5. Update head: `head = newNode;`

METHOD **Traverse and print a list**

1. Set `Node *current = head;`
2. While `current != NULL`:
   - Process `current->data`
   - Advance: `current = current->next;`

METHOD **Free an entire list**

1. While `head != NULL`:
   - `Node *tmp = head;`
   - `head = head->next;`
   - `free(tmp);`

Stacks and queues. A **stack** (LIFO) supports `push` (insert at front) and `pop` (remove from front). A **queue** (FIFO) supports `enqueue` (insert at rear) and `dequeue` (remove from front). Both can be implemented with linked lists or with arrays (circular buffer for queues).

EXEMPLE **Stack using a linked list**

`push` is exactly the front-insertion method above. `pop` removes the head node and returns its data:

```
Student pop(Node **head) {
    if (*head == NULL) return emptyStudent;
    Node *tmp = *head;
    Student data = tmp->data;
    *head = tmp->next;
    free(tmp);
    return data;
}
```

Binary trees. A **binary tree** node contains data and two pointers (`left`, `right`). A **binary search tree (BST)** maintains the invariant: all keys in the left subtree are smaller, all keys in the right subtree are larger. This enables $O(\log n)$ average-case search, insertion and deletion.

DEFINITION **Binary search tree node**

```
typedef struct TreeNode {
    Student data;
    struct TreeNode *left, *right;
} TreeNode;
```

1.9 Searching and Sorting Algorithms

Efficient data retrieval requires understanding algorithmic complexity.

PROPERTY **Linear search**

Scans elements sequentially. Time complexity: $O(n)$ worst case. Works on any array or linked list, sorted or not.

PROPERTY **Binary search**

Requires a sorted array. Repeatedly halves the search interval. Time complexity: $O(\log n)$. Not directly applicable to linked lists (no random access).

METHOD **Binary search on a sorted array of Student by matricule**

1. Set `low = 0, high = n - 1.`
2. While `low <= high`:
   - `mid = (low + high) / 2;`
   - `cmp = strcmp(key, arr[mid].matricule);`
   - If `cmp == 0` return `mid;`
   - Else if `cmp < 0` `high = mid - 1;`
   - Else `low = mid + 1;`
3. Return `-1` (not found).

PROPERTY **Sorting algorithms for GCE**

- Bubble sort: $O(n^2)$, simple, stable. Repeatedly swaps adjacent out-of-order pairs.
- Selection sort: $O(n^2)$, finds minimum and swaps to front.
- Insertion sort: $O(n^2)$ worst, $O(n)$ best (already sorted), stable, good for small/nearly-sorted data.
- Quick sort: $O(n \log n)$ average, $O(n^2)$ worst, in-place, not stable.
- Merge sort: $O(n \log n)$ worst, stable, requires $O(n)$ extra space.

EXEMPLE **Insertion sort on an array of Student by average mark**

```
void insertionSort(Student a[], int n) {
    int i, j;
    Student key;
    for (i = 1; i < n; i++) {
```

```
        key = a[i];
        j = i - 1;
        while (j >= 0 && average(a[j]) > average(key)) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
```

1.10 Text File Handling

Data typed into arrays disappears when the program ends. To store records persistently, we write them to a text file.

Mode (fopen)	Effect
`"r"`	read; fails if the file does not exist
`"w"`	write; creates the file, **erases** any previous content
`"a"`	append; writes at the end, keeps previous content
`"r+"`	read and write; file must exist
`"w+"`	read and write; creates or truncates
`"a+"`	read and append

- `FILE *f = fopen("students.txt", "a");`
- `if (f == NULL) { printf("Cannot open file"); return 1; }`
- `fprintf(f, "%s %s %.2f\n", s.matricule, s.name, s.marks[0]);`
- `fclose(f);` // always close!

Reading back uses `fscanf(f, "%s %s %f", ...)` in a loop until it returns `EOF`. Always check that `fopen` did not return `NULL`, and always `fclose` the file so buffered data is actually written to disk.

WORKED EXAMPLE 1 **Average of the best students**

Statement. Write a C program that reads the marks of n students (with n given by the user), stores them in a dynamically allocated array, and prints the class average and the highest mark.

SOLUTION

Solution.

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    int n, i;
    float *marks, sum = 0.0f, max;
    printf("Number of students: ");
    scanf("%d", &n);
    marks = malloc(n * sizeof(float));
    if (marks == NULL) return 1;
    for (i = 0; i < n; i++) {
        printf("Mark %d: ", i + 1);
        scanf("%f", &marks[i]);
    }
    max = marks[0];
    for (i = 0; i < n; i++) {
        sum += marks[i];
        if (marks[i] > max) max = marks[i];
    }
    printf("Average = %.2f, Best = %.2f\n", sum / n, max);
    free(marks);
    return 0;
}
```

Note the essential habits shown here: `&` in every `scanf`, the `NULL` check after `malloc`, and the matching `free` before the program ends.

WORKED EXAMPLE 2 **Binary search on a sorted array of structures**

Statement. Given an array `Student class[100]` sorted by `matricule`, write a function `int findStudent(Student class[], int n, char *target)` that returns the index of the student with matricule `target`, or `-1` if not found.

SOLUTION

Solution.

```
int findStudent(Student class[], int n, char *target) {
    int low = 0, high = n - 1, mid, cmp;
    while (low <= high) {
        mid = (low + high) / 2;
        cmp = strcmp(target, class[mid].matricule);
        if (cmp == 0) return mid;
        else if (cmp < 0) high = mid - 1;
        else low = mid + 1;
    }
    return -1;
}
```

This runs in $O(\log n)$ time. Precondition: the array must be sorted by `matricule` using the same string ordering as `strcmp`.

EXERCISE 1 **Student record file**

Using the `Student` structure defined above, write a program that: (a) asks the user for the matricule, name and three marks of one student; (b) appends the record to `students.txt`; (c) then reopens the file in read mode and displays all records stored so far, each with its average mark.

SOLUTION **Exercise 1**

Sketch of the solution. Declare `Student s;`. Read the fields with `scanf("%s", s.matricule);` (arrays already behave as addresses, so no `&` for strings) and `scanf("%f", &s.marks[i]);`. Open the file with `fopen("students.txt", "a")`, write with `fprintf(f,`

`"%s %s %f %f %f\n", ...)` and `fclose(f)`. Reopen with `"r"` and loop `while (fscanf(f, "%s %s %f %f %f", m, nm, &a, &b, &c) == 5)`, printing each record and `(a+b+c)/3.0f`. Close the file. Common pitfalls: forgetting `&` on the marks, comparing names with `==`, and omitting `fclose`.

EXERCISE 2 **Linked list of students**

Write a program that builds a singly linked list of `Student` records by repeatedly asking the user for a matricule, name and three marks (stop when matricule is `"END"`). Then traverse the list and print every student whose average mark is ≥ 10. Finally, free the entire list.

SOLUTION **Exercise 2**

Define `Node` with `Student data` and `Node *next`. Use `Node *head = NULL;`. Loop: read matricule into a buffer; if `strcmp(buf, "END") == 0` break; else allocate node, copy data, insert at front (`newNode->next = head; head = newNode;`). Traversal: `for (Node *p = head; p != NULL; p = p->next)` compute average, if ≥ 10 print. Freeing: `while (head) { Node *tmp = head; head = head->next; free(tmp); }`. Key points: check `malloc` return, use `strcpy` for strings, remember `free`.

KEY POINTS **Key points of this section**

- C is compiled: `source.c` → preprocessor → compiler → object file → linker → executable.
- Use `printf`/`scanf` with the right format specifiers, and never forget `&` in `scanf` for non-pointer variables.
- Distinguish = (assignment) from == (comparison); compare strings with `strcmp`.
- Functions receive copies (passing by value); pass addresses (pointers) to let a function modify a variable.
- A pointer stores an address; `malloc` obtains dynamic memory and every `malloc` needs exactly one `free`.
- `struct` (with `typedef`) models a record; arrays of structures model a whole class; text files (`fopen`, `fprintf`, `fscanf`, `fclose`) make records persistent.
- Linked lists allow dynamic size and efficient insertion/deletion; stacks (LIFO) and queues (FIFO) are specialised list disciplines.

- Binary search trees provide $O(\log n)$ average search/insert/delete when balanced.
- Linear search $O(n)$ works everywhere; binary search $O(\log n)$ requires sorted array with random access.
- Insertion sort is simple and efficient for small/nearly-sorted data; merge sort guarantees $O(n \log n)$; quick sort is fast in practice but $O(n^2)$ worst case.

2 DATA STRUCTURES, SEARCHING AND INDEXING

In the previous section we learned how to write C programs that manipulate data: variables, arrays, loops, functions, structures and dynamic memory allocation. But knowing the language is not enough. Imagine the registry of a school in Yaoundé holding the records of 10 000 students. If the records are simply thrown into memory in any order, finding one student may require examining all 10 000 records. If, however, the data is organised intelligently, the same search can take a handful of steps. **How** data is organised in memory is often more important than the program that processes it. This is the subject of data structures.

2.1 Abstract Data Types and Classification of Data Structures

DEFINITION **Data structure**

A **data structure** is a way of organising data in memory so that operations such as insert, delete and search can be performed efficiently.

It is essential to distinguish two levels of description:

- An **abstract data type** (ADT) describes what a structure does: the data it holds and the operations allowed (for example "a stack: you may push, pop, read the top"). It says nothing about how this is realised in memory.
- A **concrete implementation** describes how the ADT is realised in a programming language, for example a stack implemented with a C array and an integer `top`, or with a linked list.

The same ADT can have several implementations, each with different performance characteristics. Data structures are classified along two main axes:

Criterion	Category	Examples
Shape	Linear (elements in a sequence)	array, linked list, stack, queue
Shape	Non-linear (branching or network)	tree, graph
Memory	Static (size fixed at creation)	C array
Memory	Dynamic (grows/shrinks at run time)	linked list, dynamic stack, queue

2.2 Arrays Revisited: the Static Structure

You already know the C array: `int marks[50];` reserves one contiguous block of 50 integers. Its great strength is **direct access** (or random access): since all cells are identical and contiguous, the address of `marks[i]` is computed immediately as base + i × sizeof(int), so reading or writing any element takes constant time, written $O(1)$, whatever the position.

Its weakness is that the size is **fixed** once and for all at compile time (for static arrays) or at allocation (for VLAs/malloc'd blocks, but still fixed for the lifetime of that block). If your school admits a 51st student, an array declared for 50 cannot grow without reallocation and copying; and inserting an element in the middle forces you to shift all the following elements one cell to the right — up to n moves.

2.3 Singly Linked Lists in C

A **singly linked list** solves the size problem: each element (a **node**) is allocated separately (typically with `malloc`) and stores, next to its data, the address of the next node. The nodes no longer need to be contiguous in memory.

EXEMPLE **Node structure in C**

```
struct Node {
  int data;
  struct Node *next;
};
```

The whole list is reachable from a single pointer `head` to the first node; the last node has `next = NULL`.

Traversal. Start at `head`, follow `next` until `NULL`:

```
struct Node *p = head;
while (p != NULL) {
  printf("%d ", p->data);
  p = p->next;
}
```

Insertion at head (the cheapest insertion, $O(1)$):

1. Allocate the new node and fill its data field.
2. Set `new->next = head;` (the new node points to the old first node).
3. Set `head = new;` (the list now starts at the new node).

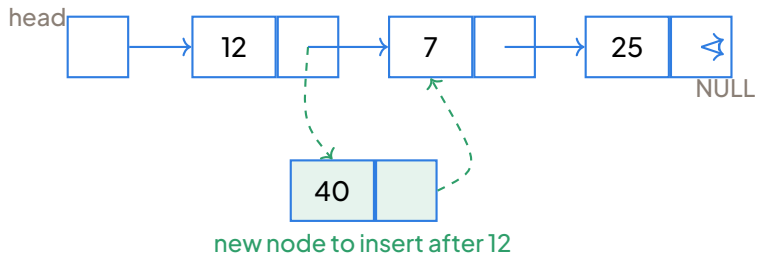
Insertion at tail requires walking to the last node, then setting `last->next = new;` and `new->next`

= NULL; — an $O(n)$ operation unless a `tail` pointer is maintained.

Deletion of a node (given a pointer to the node to delete, or searching by value) requires making the previous node point to the next one, then freeing the removed node. Special care is needed if the node to delete is the head.

EXEMPLE **Deleting a node by value (iterative)**

```
struct Node *curr = head, *prev = NULL;
while (curr != NULL && curr->data != val) {
  prev = curr;
  curr = curr->next;
}
if (curr == NULL) return; // not found
if (prev == NULL) head = curr->next; // deleting head
else prev->next = curr->next;
free(curr);
```



A singly linked list (12 → 7 → 25 → NULL). Dashed arrows: inserting node 40 after 12 only redirects two pointers — no data is moved.

WARNING **Losing the head pointer**

A classic fatal error: writing `head = new;` before `new->next = head;`. The address of the old first node is then overwritten and lost forever: the whole list becomes unreachable (a memory leak). Always link the new node to the old list first, then move `head`.

Operation	Array	Linked list
Access element i	$O(1)$ direct	$O(n)$ walk from head
Insert/delete at head	$O(n)$ (shifting)	$O(1)$
Insert/delete at tail	$O(1)$ (if space)	$O(n)$ without tail pointer, $O(1)$ with
Size	fixed at declaration	grows/shrinks freely
Memory overhead	none (compact)	one pointer per node

2.4 Stacks (LIFO) and Queues (FIFO)

A **stack** is a linear ADT where insertions and removals happen at the same end, the top: **L**ast **I**n, **F**irst **O**ut, like a pile of plates. Its core operations are:

- `push(x)`: add `x` on top.
- `pop()`: remove and return the top element (error if empty).
- `top()` or `peek()`: read the top without removing.

Applications: evaluating arithmetic expressions, the undo button of a text editor, and the **function call stack** — every time a C function calls another, the return address and local variables are pushed, and popped on return.

A **queue** is a linear ADT where insertions happen at the rear (or back) and removals at the front: **F**irst **I**n, **F**irst **O**ut, like a line of customers at a bank in Douala. Its core operations are:

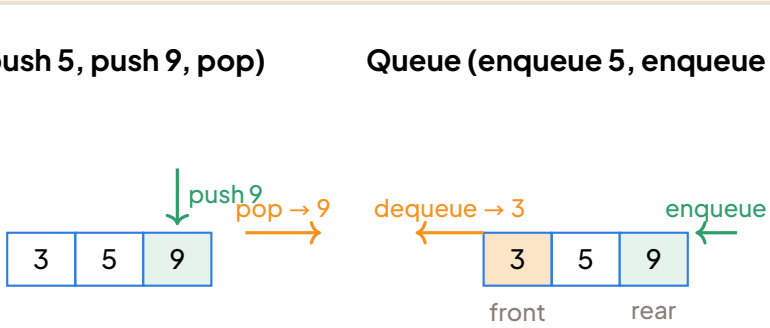
- `enqueue(x)`: add `x` at the rear.
- `dequeue()`: remove and return the front element (error if empty).
- `front()`: read the front without removing.

Applications: the printer queue (documents print in arrival order) and the **ready queue** of processes waiting for the CPU — a bridge to the operating systems part of this chapter.

PROPERTY **Stack and Queue Implementations**

Both ADTs can be implemented with arrays (static, fixed capacity) or linked lists (dynamic, unbounded).

- Array-based stack: keep an integer `top` (index of next free slot). `push` increments `top`, `pop` decrements. $O(1)$ for all ops.
- Array-based queue: use a circular buffer with indices `front` and `rear` (modulo capacity) to avoid shifting elements. $O(1)$ for all ops.
- Linked-list stack: `push` = insert at head, `pop` = delete at head. $O(1)$.
- Linked-list queue: maintain both `head` and `tail` pointers. `enqueue` at tail, `dequeue` at head. $O(1)$.



Stack: both operations at the top. Queue: enter at the rear, leave at the front.

METHOD **Choosing the right linear structure**

1. Need direct access by position and the size is known/fixed? → Array.
2. Frequent insertions/deletions, unknown or highly variable size? → Linked list.
3. Last-in-first-out behaviour (undo, function calls, expression evaluation)? → Stack.
4. First-in-first-out behaviour (waiting lines, print spooler, CPU ready queue)? → Queue.

2.5 Searching: Linear Search and Binary Search

Linear search examines the elements one by one from the first until the target is found or the array is exhausted. It works on any array, sorted or not.

EXEMPLE **Linear search in C**

```
int linear_search(int a[], int n, int key) {
  for (int i = 0; i < n; i++)
    if (a[i] == key) return i;
  return -1; // not found
}
```

EXEMPLE **Trace of linear search**

Search 7 in `[4, 9, 7, 2]`: compare with 4 (no), with 9 (no), with 7 (yes) → 3 comparisons.

Searching an absent value costs n comparisons: linear search is $O(n)$ in the worst case.

Binary search exploits sorted data (ascending order assumed): compare the key with the middle element; if it is smaller, discard the right half, otherwise discard the left half, and repeat on the remaining half.

PROPERTY **Binary search**

Binary search requires sorted data and performs at most $\lfloor \log_2 n \rfloor + 1$ comparisons: it is $O(\log n)$. Indeed each comparison halves the search interval, and an interval of size n can be halved at most $\log_2 n$ times before reaching size 1.

METHOD **Binary search algorithm (iterative)**

1. Set `low = 0, high = n-1`.
2. While `low <= high`:
   (a) `mid = low + (high - low) / 2` (avoids overflow).
   (b) If `a[mid] == key` return `mid`.
   (c) Else if `a[mid] < key` set `low = mid + 1`.
   (d) Else set `high = mid - 1`.
3. Return -1 (not found).

WORKED EXAMPLE 3 **Trace of binary search**

Search 23 in the sorted array `[3, 7, 12, 19, 23, 31, 45]` ($n = 7$).

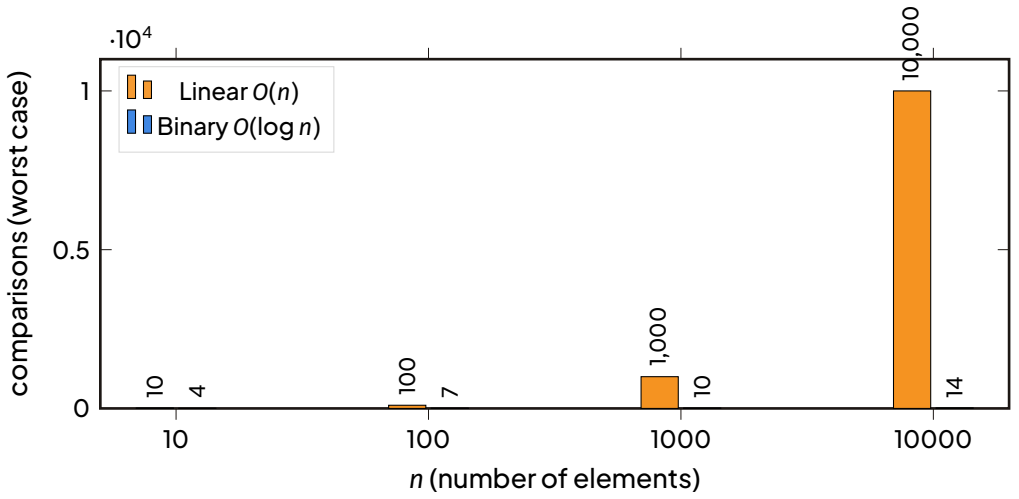
SOLUTION

1. low = 0, high = 6, mid = 3: $a[3] = 19 < 23$ → keep the right half (low = 4).
2. low = 4, high = 6, mid = 5: $a[5] = 31 > 23$ → keep the left half (high = 4).
3. low = 4, high = 4, mid = 4: $a[4] = 23$ → found at index 4.

Only 3 comparisons, and indeed $\lfloor \log_2 7 \rfloor + 1 = 3$. A linear search would have needed 5 comparisons.

WARNING **Binary search on unsorted data**

If the array is not sorted, binary search does not crash: it silently discards the half that may contain the key and returns “not found” or a wrong position — with no error message. Always check (or establish) the sorting precondition first.



Worst-case comparisons: linear search grows with n, binary search grows extraordinarily slowly (14 comparisons for 10 000 elements!).

2.6 Sorting as a Support for Searching

Since binary search is so fast, it pays to **sort** data first if we need to perform many searches. Two classical $O(n^2)$ sorting algorithms are examinable for the GCE Advanced Level.

Selection sort. Repeatedly find the smallest element of the unsorted part and swap it into its final position at the beginning of the unsorted part.

EXEMPLE **Selection sort in C**

```
void selection_sort(int a[], int n) {
  for (int i = 0; i < n-1; i++) {
    int min = i;
    for (int j = i+1; j < n; j++)
      if (a[j] < a[min]) min = j;
    // swap a[i] and a[min]
```

```
    int tmp = a[i]; a[i] = a[min]; a[min] = tmp;
  }
}
```

Bubble sort. Repeatedly scan the array, swapping any two adjacent elements that are in the wrong order; the largest values "bubble up" to the end. An optimisation stops early if a pass performs no swaps.

EXEMPLE **Bubble sort in C (optimised)**

```
void bubble_sort(int a[], int n) {
  for (int i = 0; i < n-1; i++) {
    int swapped = 0;
    for (int j = 0; j < n-1-i; j++)
      if (a[j] > a[j+1]) {
        int tmp = a[j]; a[j] = a[j+1]; a[j+1] = tmp;
        swapped = 1;
      }
    if (!swapped) break; // array already sorted
  }
}
```

Both perform about $\frac{n(n-1)}{2}$ comparisons in the worst case, hence $O(n^2)$: acceptable for a class list (e.g. $n \le 100$), slow for a national database. The winning strategy for repeated searches is: sort once ($O(n^2)$ with these algorithms, or $O(n \log n)$ with more advanced ones like QuickSort/MergeSort), then enjoy binary search at $O(\log n)$ per query.

2.7 Indexing: from Key to Address

Even sorted, a large file stored on disk is costly to scan: each disk access is thousands of times slower than a memory access. The solution used by every real database is the **index**.

DEFINITION **Index**

An **index** is an auxiliary table associating each search key with the physical position (ad-

dress, byte offset, or record number) of the corresponding record in the data file: key → address.

The index is small enough to be kept entirely in main memory (or at least its upper levels), allowing binary search (or hashing) on the index itself. Once the address is found, a single direct disk access retrieves the full record.

EXEMPLE **A matricule index for a student file**

The file `students.dat` stores records in arrival order (unsorted). We build a small sorted index in memory:

Matricule (key)	Record position (offset)
6C001	3
6C014	0
6C027	2
6C033	1

To retrieve student 6C027: binary-search the small index (4 entries → 2 comparisons), obtain position 2, then perform one single disk seek/read directly to that record — instead of scanning the whole file sequentially.

DID YOU KNOW? **Hashing — GCE enrichment**

A **hash function** computes the storage position directly from the key, without any search: for example hash(key) = key mod table_size. With a table of size 10, key 6C027 (numeric part 6027) maps to slot 6027 mod 10 = 7. Two different keys may land on the same slot: this is a **collision**, handled for example by chaining (a linked list per slot) or open addressing (probing for the next free slot). Hashing gives average-case access close to $O(1)$ and powers student information systems, mobile-money databases, and compiler symbol tables used daily in Cameroon.

KEY POINTS **Key points for the examination**

- A data structure organises data so that insert, delete and search are efficient; distinguish the ADT (what) from the implementation (how).
- Array: $O(1)$ access, fixed size. Linked list: dynamic size, $O(1)$ head insertion, $O(n)$ access — never lose `head` (memory leak).

- Stack = LIFO (push/pop); queue = FIFO (enqueue/dequeue, e.g. the CPU ready queue). Both can be array-based (circular buffer for queue) or linked.
- Linear search $O(n)$ on any data; binary search $O(\log n)$ but only on sorted data — verify the precondition!
- Selection and bubble sort are $O(n^2)$; sorting once enables fast repeated binary searches.
- An index maps key → address and retrieves a record in one disk access; hashing computes the address directly (watch for collisions).

EXERCISE 3 **Choosing and tracing**

1. A school canteen serves pupils strictly in arrival order. Which ADT models the waiting line, and why?
2. Trace binary search for the key 8 in the sorted array `[1, 3, 5, 8, 11, 13]`. Show the values of `low`, `high`, `mid` and the comparison at each step. How many comparisons?
3. Give one reason why a linked list is preferable to an array for a music playlist where songs are constantly added and removed at arbitrary positions.
4. A file contains 1 000 000 sorted records. Estimate the maximum number of comparisons needed to find a record using binary search on an in-memory index. (Hint: $\log_2 10^6 \approx 20$).

SOLUTION **Exercise 1**

1. A queue (FIFO): the first pupil to arrive is the first served — `enqueue` at the rear, `dequeue` at the front.
2. Step 1: low=0, high=5, mid=2 → a[2]=5 < 8 → low=3.
   Step 2: low=3, high=5, mid=4 → a[4]=11 > 8 → high=3.
   Step 3: low=3, high=3, mid=3 → a[3]=8 → found. 3 comparisons.
3. Insertions and removals at known positions cost $O(1)$ pointer changes (no shifting of subsequent elements), and the list grows or shrinks freely without reallocation, unlike a fixed-size array or a dynamic array where insertion in the middle is $O(n)$.
4. $\lfloor \log_2 1\,000\,000 \rfloor + 1 \approx 20$ comparisons maximum.

3 OPERATING SYSTEMS AND SYSTEM SOFTWARE

In the previous section we saw how data structures such as arrays, linked lists, trees and hash tables allow us to organise and retrieve information efficiently. An operating system (OS) is the master organiser of the whole computer: it decides which program gets the CPU, where data lives in memory, how files are stored on disk, and how devices communicate. Understanding the OS is therefore the natural next step after mastering data structures — the OS itself uses those very structures (process tables, page tables, directory trees, index blocks) to provide the services that every application relies on.

3.1 What is an Operating System?

DEFINITION **Operating System**

An **operating system** is system software that manages the hardware resources of a computer (CPU, memory, I/O devices, storage) and provides a stable, abstract interface for application programs and users. It acts as a resource manager (it allocates CPU time, memory and devices fairly), a control program (it controls the execution of programs to prevent errors and improper use) and an extended/virtual machine (it hides the messy details of the hardware behind simple services such as files and windows).

KEY POINTS **Key Roles of an OS**

- Process management – creation, scheduling, synchronisation, termination.
- Memory management – allocation, protection, virtual memory.
- File management – naming, organisation, access control, allocation methods.
- Device management – drivers, buffering, spooling.
- Security & protection – user authentication, access rights.
- User interface – command line (CLI), graphical (GUI), system calls.

DID YOU KNOW? **Operating Systems around Us**

In Cameroonian schools, cybercafés and offices you will most often meet Microsoft Windows (proprietary, GUI-centric), Linux distributions such as Ubuntu or Debian (open source, dominant on servers and for software development), and Android (Linux-based, on smartphones and tablets). All three illustrate exactly the same OS concepts studied here; only the policy details and interfaces differ.

3.2 System Software vs Application Software

DEFINITION **System Software**

System software consists of the OS kernel, utility programs (disk formatter, backup tools, antivirus), device drivers, and language processors (compilers, linkers, loaders). It runs with high privilege and directly controls the hardware on behalf of all other programs.

DEFINITION **Application Software**

Application software solves user-level problems (word processors, spreadsheets, browsers, games). It runs in user mode and requests services from the OS through **system calls** (e.g., `open`, `read`, `write`).

The **kernel** is the core of the OS: it implements process scheduling, memory management, interrupt handling and the system-call interface. Utility programs and drivers extend the kernel's functionality without being part of the kernel proper. This separation is what allows the same kernel to serve thousands of different applications.

3.3 Process Management

DEFINITION **Process**

A **process** is a program in execution together with its data, stack, heap and execution context (register values, program counter). A program is a passive file on disk; a process is the active entity that the OS schedules. Running the same program twice creates two distinct processes.

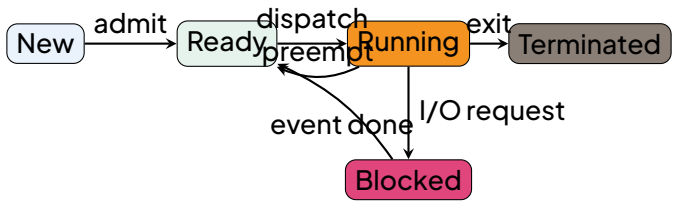
KEY POINTS **Process States and Transitions**

A process moves through the following states:

- New – being created (PCB allocated).
- Ready – waiting for the CPU.
- Running – currently executing on a CPU core.
- Blocked / Waiting – waiting for an event (I/O completion, signal).
- Terminated – finished; resources released, exit status stored.

Transitions: New → Ready (admission), Ready → Running (dispatch), Running → Ready

(preemption / time-slice expiry), Running → Blocked (I/O request, wait), Blocked → Ready (event occurs), Running → Terminated (exit or kill).



Process state-transition diagram. Note that a blocked process can never go directly to Running: it must first return to Ready and wait for the scheduler.

WARNING **Classic Exam Trap**

A process in the Blocked state cannot be dispatched directly to Running. When its event completes it moves to Ready and joins the queue like every other ready process. Drawing a Blocked → Running arrow in the state diagram is a frequent error.

DEFINITION **Process Control Block (PCB)**

The **PCB** (or Task Control Block) is the OS data structure that stores all information needed to manage a process. Typical fields:

- PID – unique process identifier.
- State – new, ready, running, blocked, terminated.
- Program Counter (PC) – address of the next instruction.
- CPU registers – accumulator, index registers, stack pointer, status flags.
- Memory management info – base/limit registers or page-table pointer.
- Accounting info – CPU time used, time limits, user ID.
- I/O status – list of open files and allocated devices.

When the CPU switches from one process to another (a **context switch**), the OS saves the running process's context into its PCB and reloads the context of the newly dispatched process from its own PCB.

3.4 CPU Scheduling

The **scheduler** decides which ready process obtains the CPU. Its goals include **fairness**, **CPU utilisation**, **throughput** (processes completed per unit time), **turnaround time** (completion − arrival), **waiting time** (turnaround − burst) and **response time** (first response − arrival).

KEY POINTS **Scheduling Terminology**

- Burst time – CPU time the process needs.
- Arrival time – when the process enters the ready queue.
- Preemptive algorithm – the OS may take the CPU away (e.g., Round Robin).
- Non-preemptive algorithm – a running process keeps the CPU until it finishes or blocks (e.g., FCFS, basic SJF).

We study three classic algorithms by hand, drawing **Gantt charts** to visualise the timeline.

METHOD **Solving a Scheduling Exercise**

1. List all processes with arrival time and burst time.
2. Apply the algorithm's rule to decide the execution order; draw the Gantt chart.
3. For each process compute:
   - Completion time (CT) – end of its last CPU burst.
   - Turnaround time (TAT) = CT − Arrival.
   - Waiting time (WT) = TAT − Burst.
4. Average WT and average TAT are the sums divided by the number of processes.

WARNING **Common Mistake**

In GCE questions, turnaround time and waiting time are often confused. Remember: turnaround = total time spent in the system (waiting + execution); waiting = time spent only in the ready queue. Always compute both explicitly in a table.

First-Come First-Served (FCFS). Non-preemptive; processes run strictly in arrival order. Simple to implement (a FIFO queue), but it suffers from the convoy effect: one long process makes many short processes wait behind it, raising the average waiting time.

Shortest Job First (SJF). Chooses the ready process with the smallest next CPU burst. The non-preemptive version lets the chosen process run to completion; the preemptive version is called Shortest

Remaining Time First (SRTF).

PROPERTY **Optimality of SJF**

For a given set of processes with known burst times, SJF yields the minimum average waiting time among all scheduling algorithms. Intuition (exchange argument): if a long job runs before a shorter one, swapping them reduces the waiting time of the shorter job by more than it increases that of the longer one, so any non-SJF order can be improved. (The justification is examinable; a formal proof is not required.)

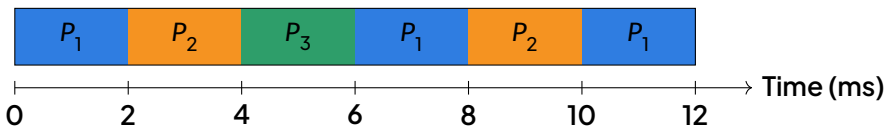
Round Robin (RR). Preemptive; each process receives a fixed **time quantum** q. If its burst exceeds q, the process is preempted and placed at the tail of the ready queue. RR is designed for time-sharing systems: a small q gives good response time but increases context-switch overhead; a very large q degenerates into FCFS.

WORKED EXAMPLE 4 **Round Robin Scheduling Example**

Statement: Three processes arrive at time 0 with burst times: $P_1 = 6$, $P_2 = 4$, $P_3 = 2$ (all in ms). Time quantum $q = 2$. Draw the Gantt chart and compute the average waiting and turnaround times.

SOLUTION

Solution. The ready queue evolves as follows: P_1 runs (0–2), P_2 runs (2–4), P_3 runs (4–6) and finishes, P_1 resumes (6–8), P_2 resumes (8–10) and finishes, P_1 resumes (10–12) and finishes.



Gantt chart for Round Robin with $q = 2$.

Process	Arrival	Burst	CT	TAT = CT − Arr	WT = TAT − Burst
P_1	0	6	12	12	6
P_2	0	4	10	10	6
P_3	0	2	6	6	4

Average waiting time = (6 + 6 + 4)/3 = 5.33 ms. Average turnaround time = (12 + 10 + 6)/3 = 9.33 ms.

3.5 Memory Management

DEFINITION **Logical vs Physical Address**

A **logical address** (also called virtual address) is generated by the CPU when a program executes; a **physical address** is the actual location in RAM. The Memory Management Unit (MMU) translates logical addresses into physical addresses at run time, so a program does not need to know where it is loaded.

In **contiguous allocation**, each process occupies a single contiguous block of physical memory. The OS maintains a list of free **partitions** (holes). Two fragmentation problems arise:

- **Internal fragmentation** – the allocated partition is larger than the request; the leftover space inside the partition is wasted.
- **External fragmentation** – free memory is split into many small holes; the total free space may be sufficient for a request, but no single hole is large enough.

DEFINITION **External Fragmentation**

External fragmentation occurs when free memory is divided into small non-contiguous holes that cannot satisfy a large request, even though the sum of the hole sizes is sufficient. It can be reduced by compaction (moving processes together), which is expensive, or avoided entirely by paging.

Placement strategies. When a process of size s arrives, the OS selects a hole according to one of these policies:

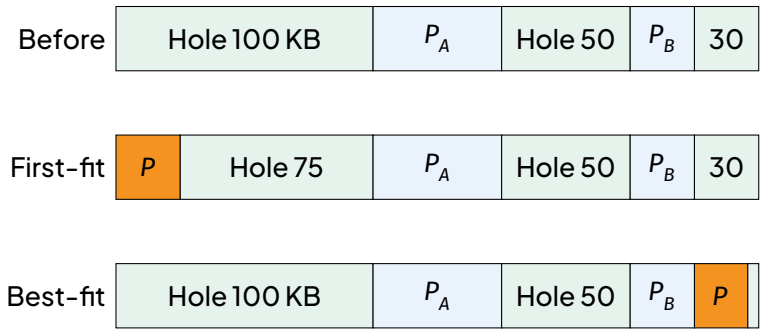
- **First-fit** – the first hole of size $\geq s$ (fast; tends to leave large holes at the high end of memory).
- **Best-fit** – the smallest hole of size $\geq s$ (minimises the leftover fragment, but creates many tiny, unusable holes).
- **Worst-fit** – the largest hole (leaves a large remainder, but consumes the biggest holes quickly).

EXEMPLE **First-fit vs Best-fit vs Worst-fit**

Memory contains free holes of sizes 100, 50, 30 KB (in address order). A process P of 25 KB arrives.

- First-fit takes the 100 KB hole → remaining holes: 75, 50, 30.
- Best-fit takes the 30 KB hole → remaining holes: 100, 50, 5.
- Worst-fit takes the 100 KB hole → remaining holes: 75, 50, 30.

Best-fit leaves a 5 KB hole that is almost useless — a typical source of external fragmentation.



Placement of a 25 KB process *P*. First-fit splits the first adequate hole (100 KB → 25 + 75). Best-fit splits the smallest adequate hole (30 KB → 25 + 5), leaving a tiny 5 KB fragment.

DID YOU KNOW? **Modern Solution – Paging**

Modern OSs (Linux, Windows, Android) use **paging**: physical memory is divided into fixed-size frames (commonly 4 KB) and each process into pages of the same size. A page table maps logical pages to physical frames, so a process no longer needs contiguous memory: external fragmentation disappears. This is an enrichment remark — for the GCE you must know that paging is the standard answer to the fragmentation problem.

3.6 File Management

A **file system** organises data on secondary storage (HDD, SSD, flash) into named files and directories. The OS provides system calls for create, open, read, write, close, delete, and directory operations.

KEY POINTS **File Allocation Methods**

- Contiguous – the file occupies consecutive blocks; simple and fast for sequential access, but suffers external fragmentation and makes file growth difficult.
- Linked – each block contains a pointer to the next; no external fragmentation, but random access is slow (one must follow the chain of links).
- Indexed – an index block holds pointers to all the blocks of the file; supports direct (random) access and easy growth. This is the disk-level counterpart of the indexing module built in the C project (Lesson 95): a B-tree or hash index on disk.

Directories map human-readable names to file metadata (inode number, attributes). A hierarchical (tree) directory structure is standard; each directory is itself a file containing `(name, inode)` pairs.

3.7 Deadlocks (Brief Overview)

A **deadlock** is a permanent blocking of a set of processes, each waiting for a resource held by another process in the set. Four **necessary conditions** (Coffman conditions) must hold simultaneously:

1. **Mutual exclusion** – at least one resource is non-shareable.
2. **Hold and wait** – a process holds resources while requesting others.
3. **No preemption** – resources cannot be forcibly taken away.
4. **Circular wait** – a cycle of processes $P_1 \to P_2 \to \ldots \to P_n \to P_1$, each waiting for a resource held by the next.

KEY POINTS **Deadlock Handling Strategies**

- Prevention – design the system so that at least one condition can never hold (e.g., require all resources at once to break Hold-and-Wait).
- Avoidance – make dynamic decisions (e.g., the Banker's algorithm) to keep the system in a safe state.
- Detection & recovery – allow deadlocks, periodically detect cycles in the resource-allocation graph, then abort or preempt processes.
- Ostrich algorithm – ignore the problem (used by many general-purpose OSs because deadlocks are rare and prevention is costly).

3.8 Summary and Key Points

KEY POINTS **Essential Takeaways for the GCE**

- OS = resource manager + control program + virtual machine; kernel, utilities, drivers.
- Process = active program; the PCB holds its context; the state diagram (with correct transitions) is examinable.
- Scheduling: FCFS (simple, convoy effect), SJF (optimal average waiting time), RR (time quantum, good response time). Always draw the Gantt chart first, then compute CT, TAT, WT.
- Memory: logical vs physical addresses; contiguous allocation leads to internal/external fragmentation; first-fit / best-fit / worst-fit; paging eliminates external fragmentation.
- File allocation: contiguous, linked, indexed (linked to the indexing project of Lesson

95).

- Deadlock: four necessary conditions; prevention, avoidance, detection & recovery.

EXERCISE 4 **Practice Exercise**

1. Five processes arrive at time 0 with burst times (ms): $P_1 = 10$, $P_2 = 1$, $P_3 = 2$, $P_4 = 1$, $P_5 = 5$. Draw the Gantt chart and compute the average waiting time for (a) FCFS (order $P_1, \dots, P_5$), (b) non-preemptive SJF, (c) RR with $q = 1$.
2. A memory has free holes of sizes 40, 20, 60, 30 KB (in address order). A process of 25 KB arrives. Give the resulting hole list after (a) first-fit, (b) best-fit, (c) worst-fit. In which case(s) would a later 50 KB request fail even though more than 50 KB is free in total?
3. Explain why indexed allocation is preferred for a database file that requires frequent random record access.

SOLUTION **Exercise 1**

(a) FCFS order P_1, P_2, P_3, P_4, P_5: completion times $10, 11, 13, 14, 19$; waiting times $0, 10, 11, 13, 14$; average WT $= 48/5 = 9.6$ ms.
(b) SJF order P_2, P_4, P_3, P_5, P_1: completion times $1, 2, 4, 9, 19$; waiting times $0, 1, 2, 4, 9$; average WT $= 16/5 = 3.2$ ms.
(c) RR with $q = 1$: execution sequence $P_1, P_2, P_3, P_4, P_5, P_1, P_3, P_5, P_1, P_5, P_1, P_5, P_1, P_5$, then P_1 alone for its last 5 quanta (total 19 quanta). Completion times: $P_2 = 2$, $P_4 = 4$, $P_3 = 7$, $P_5 = 14$, $P_1 = 19$. Waiting times: $P_1 = 9$, $P_2 = 1$, $P_3 = 5$, $P_4 = 3$, $P_5 = 9$; average WT $= 27/5 = 5.4$ ms.

SOLUTION **Exercise 2**

(a) First-fit uses the 40 KB hole → splits into 25 KB allocated + 15 KB free. New holes: 15, 20, 60, 30.
(b) Best-fit uses the 30 KB hole → splits into 25 KB allocated + 5 KB free. New holes: 40, 20, 60, 5.
(c) Worst-fit uses the 60 KB hole → splits into 25 KB allocated + 35 KB free. New holes: 40, 20, 35, 30.
In all three cases the total free space is 125 KB but scattered. A 50 KB request fails in cases (b) and (c) (largest holes 40 KB), although 125 KB is free: this is exactly external fragmentation. In case (a) the 60 KB hole still satisfies the request.

SOLUTION **Exercise 3**

Indexed allocation stores all block pointers of the file in an index block (or a B-tree). Random access to record k requires only one index lookup plus one data-block read — $O(1)$ with a flat index, $O(\log n)$ with a B-tree. Contiguous allocation makes file growth painful (reallocation and copying), while linked allocation forces traversal of k links for record k. Hence indexed allocation — the same idea as the indexing module of Lesson 95 — is the natural choice for database files.

4 HANDS-ON PROJECTS: INDEXING MODULE AND OS SIMULATOR

In the previous section we studied operating systems from the outside: what scheduling, memory management and system software do. In this final section we roll up our sleeves and build. You will develop two complete C projects that bring together everything learnt in this chapter: a **data indexing and search module** (which uses structures, files, sorting and searching) and a **CPU and memory allocation simulator** (which imitates the scheduler and the memory manager of a real operating system). These are exactly the kind of projects examiners expect at the GCE Advanced Level practical examination: modest in size, but rigorously analysed, modular, tested and documented.

4.1 Project Methodology: From Idea to Working Program

A project is not "a lot of code typed quickly". A project is a process. Professional programmers — and GCE examiners — follow the same stages:

1. **Analysis.** Before writing a single line, answer three questions: What are the **inputs** (data the program receives)? What are the **outputs** (results it must produce)? What **processing** transforms inputs into outputs? Write the answers on paper.
2. **Modular design.** Decompose the problem into **modules**, one module per feature. In C, each module typically lives in its own file: a header file (`.h`) containing structure definitions and function prototypes, and an implementation file (`.c`) containing the function bodies. The `main()` function should only orchestrate the modules, usually through a menu.
3. **Coding.** Implement the modules one at a time, in a sensible order (data structures first, then input/output, then processing, then the menu).
4. **Testing.** Execute the program against a written **test plan** and compare actual results with expected results.
5. **Documentation.** Write the report: problem statement, design choices, code listing, test results, difficulties encountered.

METHOD **Developing a project in stages**

1. Choose the smallest useful feature (for example: “read the records from the file and display them”).
2. Make it work completely: it compiles, it runs, it is tested.
3. Save a backup copy of this working version (for example `project_v1.c`) before touching anything.
4. Add the next feature, test again, back up again (`project_v2.c`), and so on.
5. If a new feature breaks the program, you can always return to the last working backup instead of starting from zero.

WARNING **Common mistake: everything in main()**

Writing all the code inside `main()` produces a single block of hundreds of lines that is impossible to debug: you cannot test one feature in isolation, you cannot reuse code, and a single error hides among unrelated instructions. Decompose into functions: one function per task (`addRecord`, `sortRecords`, `binarySearch`, …). A good rule of thumb: no function should be longer than one screen.

METHOD **Building a test plan**

For each feature of the project, build a small table with three families of cases:

1. Normal cases: typical, expected input (e.g. search for a matricule that exists).
2. Boundary cases: input at the limits (e.g. empty file, first or last record, table completely full).
3. Error cases: invalid input (e.g. letters where a number is expected, a menu choice outside the range).

For every case, write the expected result before running the program; then run it and compare. A feature is “finished” only when all its cases pass.

4.2 Project 1: Data Indexing & Search Module

Specification. A school (think of a college in Bamenda or Yaoundé) keeps its students’ results in a text file. Each line of the file contains one record: matricule (a unique integer), name, class and average. Example file `students.txt`:

Matricule	Name	Class	Average
1023	Amina Bello	Tle C	14.50
1017	John Tabi	Tle C	11.25
1042	Marie Ngo	Tle D	15.75

The program must offer a menu allowing the user to: **add** a record, **display** all records, **sort** the records by matricule, **build an index array** (an array of positions in matricule order), **search by matricule** using binary search on the sorted index, and **search by name** using linear search.

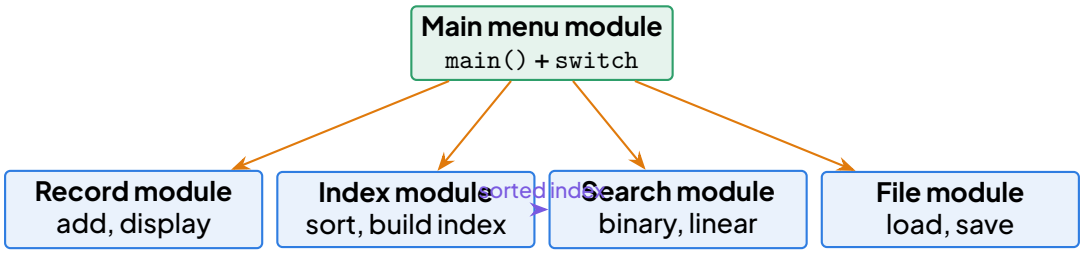
Design decisions. The natural data structure is a structure:

- `typedef struct {`
- `  int matricule;`
- `  char name[40];`
- `  char className[10];`
- `  float average;`
- `} Student;`

We choose an array of structures rather than a linked list. Why? The number of students is bounded and known approximately; arrays give direct access by position, which binary search requires (binary search on a linked list is useless because reaching the middle element costs a full traversal). A linked list would be preferable only if insertions and deletions in the middle were very frequent — not our case here.

The **index array** is a second array `index[]` where `index[i]` holds the position of the i-th student in matricule order. Sorting the index instead of the records avoids moving whole structures in memory — this is exactly the principle of index files used by real database systems.

The interface is menu-driven: a loop displays the options, reads the choice, and a `switch` statement dispatches to the right function.



Modular architecture of Project 1: the main menu delegates every task to a specialised module; the search module relies on the index built by the index module.

Testing Project 1. The boundary and error cases are decisive:

Feature	Case	Expected result
Load	File does not exist	Start with an empty table, no crash
Add	Duplicate matricule	Refuse the record with a clear message
Add	Average = -5 or 25	Reject: average must be in [0, 20]
Binary search	Matricule present (first, middle, last)	Record displayed
Binary search	Matricule absent	"Not found" message
Linear search	Empty table	"Not found" message, no crash
Menu	User types a letter	Menu re-displayed, no infinite loop

WARNING **Validating user input**

`scanf` leaves unread characters in the keyboard buffer when the user types a letter instead of a number, which often causes an infinite menu loop. Always check the return value of `scanf` (it returns the number of items successfully read) and flush the buffer on failure. Never trust the user: validate ranges (average between 0 and 20, menu choice between 1 and 7) before using the value.

4.3 Project 2: CPU & Memory Allocation Simulator

DEFINITION **Simulator**

A **simulator** is a program that imitates the behaviour of a real system in order to study it without using the real system. A flight simulator trains pilots without an aeroplane; our CPU simulator lets us compare scheduling algorithms without modifying a real operating system.

Specification. The program reads from a text file a list of processes, each described by: PID (process identifier), arrival time, burst time (CPU time needed) and memory size. It must then:

1. Simulate **FCFS** (First-Come, First-Served) and **Round Robin** (with a quantum q given by the user) scheduling;
2. Display the **Gantt chart** in text mode;
3. Compute and display the **average waiting time** and **average turnaround time**;
4. Simulate a **fixed-size memory** divided into partitions, allocating each process with **first-fit** and then **best-fit**, and display the **memory map** and the **fragmentation** after each allocation.

Data structures. Each process is described by a Process Control Block (PCB), exactly as in a real OS:

```
typedef struct {
 int pid, arrival, burst, remaining, memSize;
 int waiting, turnaround, done;
} PCB;
```

The field `remaining` (burst time still to execute) is essential for Round Robin. The **ready queue** can be implemented as a linked list (easy insertion/removal) or as a circular array (simpler, fixed size): both are acceptable; justify your choice in the report. Memory partitions are an array of structures `{start, size, free, pid}`.

WARNING **Common mistake in Round Robin**

The most frequent error is to forget to re-queue a process whose burst is not finished when its quantum expires. Correct behaviour: when a process has consumed its quantum and `remaining > 0`, it leaves the CPU and goes to the back of the ready queue; newly arrived processes during that quantum must join the queue before it. Forgetting the re-queue silently turns Round Robin into FCFS — and all your average times become wrong.

WORKED EXAMPLE 5 **FCFS and Round Robin on the same data**

Three processes: P1 (arrival 0, burst 8), P2 (arrival 1, burst 4), P3 (arrival 2, burst 2). Compute the average waiting time with FCFS and with Round Robin ($q = 3$).

SOLUTION

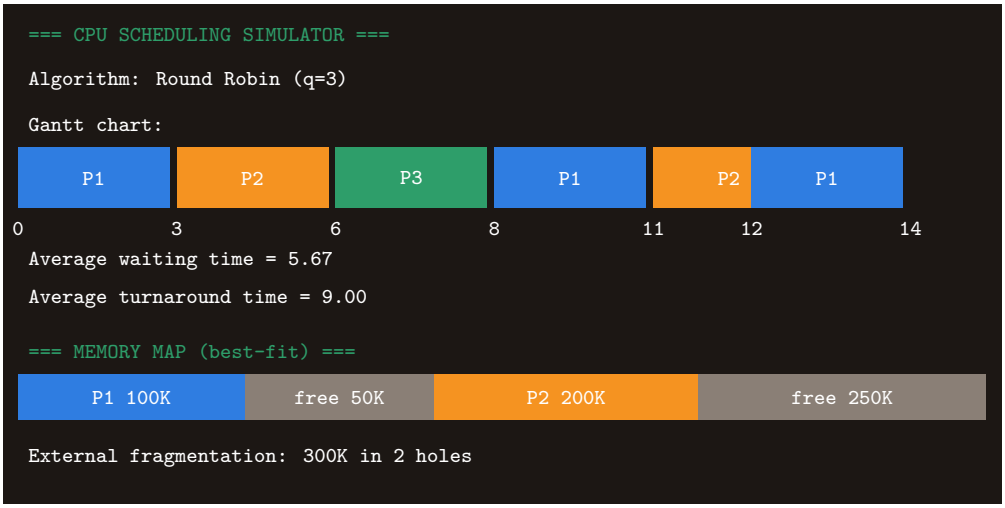
FCFS. Order of arrival: P1, P2, P3. Gantt: P1 runs $[0, 8]$, P2 runs $[8, 12]$, P3 runs $[12, 14]$.

$$W_1 = 0, \quad W_2 = 8 - 1 = 7, \quad W_3 = 12 - 2 = 10$$
$$\bar{W} = \frac{0 + 7 + 10}{3} = \frac{17}{3} \approx 5.67$$

Round Robin, $q = 3$. Timeline: P1 $[0, 3]$ (rem. 5), P2 $[3, 6]$ (rem. 1), P3 $[6, 8]$ (finished), P1 $[8, 11]$ (rem. 2), P2 $[11, 12]$ (finished), P1 $[12, 14]$ (finished).

$$W_1 = (14 - 0 - 8) = 6, \quad W_2 = (12 - 1 - 4) = 7, \quad W_3 = (8 - 2 - 2) = 4$$
$$\bar{W} = \frac{6 + 7 + 4}{3} = \frac{17}{3} \approx 5.67$$

(Recall: waiting time = completion − arrival − burst.) Here the averages coincide, but the response time of the short process P3 is much better under Round Robin (it first runs at time 6 instead of 12) — a point worth discussing in the report.



Mock console output of the simulator: text-mode Gantt chart with time markers, average times, and memory map showing allocated partitions, free holes and external fragmentation.

Memory allocation. With first-fit, each process is placed in the first free hole large enough; with best-fit, in the smallest hole large enough. After each allocation, print the memory map (as in the figure) and compute the **external fragmentation**: the total free memory scattered in holes too small or too dispersed to be used. Running both strategies on the same data and comparing fragmentation is an excellent discussion point for the oral defence.

4.4 Presentation and Evaluation

The **project report** should contain, in order: the problem statement (in your own words), the analysis (inputs / outputs / processing), the design decisions with their justifications (why an array, why binary search, why a circular queue), the commented code listing, the test plan with its results, and an honest conclusion (difficulties, possible improvements). For the oral defence, rehearse a five-minute demonstration: compile, run on the examiner's data, explain one function of your choice.

KEY POINTS **GCE exam tip**

In the practical paper, marks are awarded for: correct compilation, correct results on the examiner's test data, readability (meaningful names, indentation) and comments explaining each module — not for program length. A short, clear, correct program always scores higher than a long, obscure one. Test with the examiner's perspective in mind: empty file, absent key, extreme values.

EXERCISE 5 **Consolidation**

1. Write the test plan (normal, boundary, error cases) for the sort by matricule feature of Project 1.
2. For the processes P1 (arrival 0, burst 5), P2 (arrival 1, burst 3), P3 (arrival 2, burst 6), draw the Gantt chart and compute the average waiting time for FCFS and for Round Robin with $q = 2$.
3. Memory has holes of 100K, 500K, 200K and 300K (in that order). A process of 212K arrives. Where is it placed with first-fit? With best-fit? Justify.

SOLUTION **Consolidation**

1. Normal: several unsorted records → ascending matricule order. Boundary: single record (unchanged); already sorted file; two records. Error: empty table → message, no crash; duplicate matricules should have been refused at insertion. 2. FCFS: P1 [0, 5], P2 [5, 8], P3 [8, 14]; $W = (0 + 4 + 6)/3 = 10/3 \approx 3.33$. RR ($q = 2$): P1 [0, 2], P2 [2, 4], P3 [4, 6], P1 [6, 8], P2 [8, 9], P3 [9, 11], P1 [11, 12], P3 [12, 14]; completions 12, 9, 14; $W = ((12 - 0 - 5) + (9 - 1 - 3) + (14 - 2 - 6))/3 = (7 + 5 + 6)/3 = 6$. 3. First-fit: the 500K hole (first hole ≥ 212K), leaving 288K. Best-fit: the 300K hole (smallest hole ≥ 212K), leaving 88K — best-fit wastes less on this allocation.

DID YOU KNOW? **Why simulators matter**

Before a new scheduling algorithm is ever installed in a real operating system, it is tested for months on simulators fed with recorded workloads. The same idea is used in Cameroon and worldwide to simulate traffic in Douala, the spread of epidemics, or the level of the Sanaga river behind a dam: when experimenting on the real system is too costly, too slow or too dangerous, we simulate. Your modest Project 2 uses exactly the same scientific method.

5 INTEGRATION ACTIVITY

5.1 Aims of the activity

This integration activity closes the chapter by mobilising, in a single realistic project, all the competences developed in Lessons 93 to 96. At the end of the activity, you should be able to:

- design a **structure** in C to model real-world data (a student record) and store a collection of records in a **file**;
- implement a **sorting algorithm** and build an **index** on a key field (the matricule);
- code **linear search** and **binary search**, count their comparisons and compare their efficiency experimentally;
- model the workload of a real computer as a set of **processes** and simulate **FCFS** and **Round Robin** scheduling, by hand and with a C program;
- interpret performance measures (average waiting time, average turnaround time) and justify a technical recommendation in a short written report.

KEY POINTS **Skills mobilised**

C programming (structures, files, functions), data structures (arrays, index tables), searching and sorting, process scheduling (FCFS, Round Robin), performance analysis, teamwork and scientific writing.

5.2 Situation

Digitalising the Registrar's Office of Government High School Nkol-Eton.

Government High School Nkol-Eton, in the Yaoundé area, has about 1200 students. Until now, the registrar's office keeps student records (matricule, name, class, fees status) in large paper registers. Finding one student's file can take several minutes, and errors are frequent during the examination period.

The Principal has bought **one single computer** for the office and asks your Computer Science club to help in two ways:

1. **Build a prototype records program in C.** The program must store student records, sort them, build an index on the matricule, and answer queries of the form: "give me the record of student with matricule M".
2. **Explain the slowdowns of the office computer.** The secretary complains that the machine "freezes" when she runs several tasks together. The club must model the workload as processes, simulate the CPU scheduler, and recommend a scheduling policy and a time quantum.

Data for Part A (records program). For the prototype, use the following 8 sample records (the

real system will hold 1200; your code must therefore be efficient). The records are initially stored in arrival order, not sorted by matricule.

Matricule	Name	Class	Fees paid (FCFA)
GHS-1042	Amina Bello	Tle C	50000
GHS-1017	Ndi Che	Tle A	50000
GHS-1088	Fotso Larissa	Tle D	25000
GHS-1005	Etoundi Paul	Tle C	50000
GHS-1063	Kamga Brice	Tle A	0
GHS-1030	Mbappe Sonia	Tle D	50000
GHS-1091	Tchoupo Eric	Tle C	25000
GHS-1024	Ngassa Claire	Tle A	50000

For simplicity in C, store the numeric part of the matricule only (e.g. 1042) as an integer key.

Data for Part B (scheduling simulation). At a typical busy moment, four programs run on the office computer:

Process	Arrival time (ms)	Burst time (ms)
P_1 Word processor (typing report cards)	0	8
P_2 Printing (class lists)	1	4
P_3 Antivirus scan	2	9
P_4 Spreadsheet (fees balance)	3	5

The computer has a **single CPU**. You will compare FCFS and Round Robin (with quantum $q = 2$ ms and $q = 4$ ms) using the average waiting time and average turnaround time.

Deliverable. Each team of 3–4 students submits a short report (3–5 pages) containing: the C code, the trace tables and Gantt charts, the measured comparison counts, and a justified recommendation to the Principal.

5.3 Tasks

Part A — The records program.

1. Define a C structure `Student` with fields: `matricule` (int), `name` (string), `class_name` (string), `fees` (int). Declare an array of 8 students and initialise it with the data above (or read it from a text file `students.txt`).
2. Write a function that sorts the array by matricule (use selection sort or bubble sort). Show the state of the array after sorting.
3. **Indexing.** Explain why, for 1200 records on disk, one would build an **index**: a separate sorted table of pairs (matricule, position). Build such an index for the 8 records and draw it.
4. Code `linearSearch(tab, n, key)` and `binarySearch(tab, n, key)`, each returning the position found (or -1) and counting the number of key comparisons in a global counter.

5. Run both searches for matricule 1091 (present) and 1050 (absent). Record the comparison counts in a table. What do you observe? What would happen with $n = 1200$ records (compute $\log_2 1200 \approx 10.2$)?
6. Why does binary search require the array (or index) to be sorted? What is the cost of keeping it sorted when new students register?

Part B — The scheduling simulation.

7. Simulate **FCFS** by hand: draw the Gantt chart, then compute the waiting time and turnaround time of each process, and the two averages.
8. Simulate **Round Robin with** $q = 2$ by hand: Gantt chart, individual and average waiting and turnaround times.
9. Repeat with $q = 4$. Compare the three policies in a summary table.
10. Write (or complete) a small C program that reads the 4 processes and simulates Round Robin for any quantum q entered by the user. Test it with $q = 1, 2, 3, 4, 6$.
11. **Recommendation.** In 10–15 lines, advise the Principal: which policy and which quantum give the most responsive system for the secretary, and why? Mention at least one drawback of a quantum that is too small or too large.

5.4 Model answer

Part A.

1. Structure and storage.

```
typedef struct {
  int matricule;
  char name[30];
  char class_name[10];
  int fees;
} Student;
Student tab[8];
FILE *f = fopen("students.txt", "r");
for (i = 0; i < 8; i++)
  fscanf(f, "%d %s %s %d", &tab[i].matricule,
         tab[i].name, tab[i].class_name, &tab[i].fees);
fclose(f);
```

2. Sorting by matricule (selection sort, key = matricule) gives:

Matricule	Name	Class	Fees
1005	Etoundi Paul	Tle C	50000
1017	Ndi Che	Tle A	50000
1024	Ngassa Claire	Tle A	50000
1030	Mbappe Sonia	Tle D	50000
1042	Amina Bello	Tle C	50000
1063	Kamga Brice	Tle A	0
1088	Fotso Larissa	Tle D	25000
1091	Tchoupo Eric	Tle C	25000

3. Index. Reading 1200 records from disk in arrival order forces a slow linear scan for every query. An **index** is a small sorted table of pairs (key, position) kept in memory; searching the index (fast, binary search) directly gives the position of the record. For our sorted array, the index is simply:

Key	1005	1017	1024	1030	1042	1063	1088	1091
Position	0	1	2	3	4	5	6	7

4. Searches with comparison counters.

```
int linearSearch(Student t[], int n, int key) {
  for (int i = 0; i < n; i++) {
    comparisons++;
    if (t[i].matricule == key) return i;
  }
  return -1;
}
int binarySearch(Student t[], int n, int key) {
  int lo = 0, hi = n - 1, mid;
  while (lo <= hi) {
    mid = (lo + hi) / 2;
    comparisons++;
    if (t[mid].matricule == key) return mid;
    if (t[mid].matricule < key) lo = mid + 1;
    else hi = mid - 1;
  }
```

```
    return -1;
}
```

5. Measured comparisons.

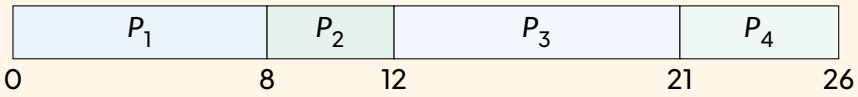
Search	Key 1091 (present)	Key 1050 (absent)
Linear (sorted array)	8	8
Binary (sorted array)	2	3

Binary search is dramatically faster: with $n = 1200$, linear search needs up to 1200 comparisons, while binary search needs at most $\lceil \log_2 1200 \rceil = 11$.

6. Binary search halves the search interval at each step, which is only valid if the keys are ordered. The price to pay: every new registration must be inserted at the correct position (or the index rebuilt), so insertions are costlier than simple appends — a classic space/time trade-off.

Part B.

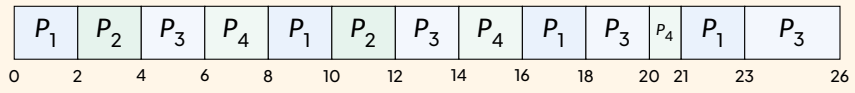
7. FCFS (order of arrival P_1, P_2, P_3, P_4):



Gantt chart for FCFS (times in ms).

Process	Arrival	Burst	Waiting	Turnaround
P_1	0	8	0	8
P_2	1	4	7	11
P_3	2	9	10	19
P_4	3	5	18	23
Average			**8.75**	**15.25**

8. Round Robin, $q = 2$. Ready queue evolution gives the chart:



Gantt chart for Round Robin with $q = 2$ ms.

Completion times: P_1 at 23, P_2 at 12, P_3 at 26, P_4 at 21.

Process	Completion	Turnaround	Waiting
P_1	23	23	15
P_2	12	11	7
P_3	26	24	15
P_4	21	18	13
Average		**19.0**	**12.5**

9. Round Robin, $q = 4$ gives completion times $P_1 : 26$, $P_2 : 9$, $P_3 : 25$, $P_4 : 18$; average turnaround 18.25 ms, average waiting 11.75 ms. Summary:

Policy	Avg. waiting (ms)	Avg. turnaround (ms)
FCFS	8.75	15.25
RR, $q = 2$	12.50	19.00
RR, $q = 4$	11.75	18.25

10. The C simulator uses a queue of ready processes; at each step it runs the head of the queue for at most q ms, enqueues newly arrived processes, then re-enqueues the unfinished process. Testing $q = 1, 2, 3, 4, 6$ confirms the hand results and shows that as q grows, RR tends towards FCFS.

11. Recommendation. Although FCFS gives the best averages here, it makes the word processor wait behind a long antivirus scan: the secretary perceives a "frozen" machine. We therefore recommend **Round Robin with** $q = 4$ **ms**: every program receives CPU time regularly, so the screen stays responsive, and the averages remain close to FCFS. A quantum that is too small ($q = 1$) wastes time in context switches; a quantum that is too large behaves like FCFS and reintroduces the freezing problem.

WARNING **Common mistakes**

Forgetting that a process arriving during a quantum joins the queue before the preempted process is re-enqueued; confusing waiting time with turnaround time; using binary search on unsorted data; comparing matricules as strings instead of integers.

6 METHODS AND WORKED EXAMPLES

6.1 Skill 1 — Declaring and Manipulating a C Structure (struct)

METHOD **Defining and using a struct in C**

To group several related pieces of data (a record) into one type:

1. Declare the structure before `main`: `struct Student { char name[30]; int age; float avg; };`
2. Declare variables of that type: `struct Student s1;` or an array `struct Student class[50];`
3. Access a field with the dot operator: `s1.age = 17;`
4. Read a string safely with `scanf("%29s", s1.name);` (no `&` before an array name; limit the width to avoid overflow).
5. Pass to a function by pointer when the function must modify the record: `void update(struct Student *s){ s->avg = 15.5; }` — note the arrow `->` used with pointers.

Reflexes: every `struct` declaration ends with a semicolon after the closing brace; `float` is printed with `%f`; always `#include <stdio.h>` for `printf`/`scanf`.

WORKED EXAMPLE 6 **GCE-style: a student record**

A school in Bamenda stores, for each Upper Sixth student, a name, an age and an average mark. (a) Define a C structure `Student` for this record. (b) Write a program that reads one student's data from the keyboard and displays it. (c) State the output if the user enters `NDIFOR`, `18`, `14.5`.

SOLUTION

(a) Structure definition. We group the three fields into one type:

```
struct Student {
    char name[30];
    int age;
    float avg;
};
```

(b) Complete program.

```
#include <stdio.h>
struct Student { char name[30]; int age; float avg; };
int main(void) {
  struct Student s;
  printf("Name: "); scanf("%29s", s.name);
  printf("Age: "); scanf("%d", &s.age);
  printf("Average: "); scanf("%f", &s.avg);
  printf("%s, age %d, average %.1f\n", s.name, s.age, s.avg);
  return 0;
}
```

(c) Output: `NDIFOR, age 18, average 14.5` (the `%.1f` format rounds to one decimal place). Examiner's remark: the most common errors are forgetting the semicolon after the structure's closing brace, writing `scanf("%s", &s.name)` (the `&` is wrong for an array), and using `%d` to read a `float` instead of `%f`.

6.2 Skill 2 — Sequential Search on an Array

METHOD **Linear search**

To find a value x in an unsorted array `T` of n elements:

1. Initialise an index `i = 0` and a position variable `pos = -1` (meaning "not found").
2. Scan: `while (i < n && pos == -1)` — if `T[i] == x`, set `pos = i`; otherwise increment `i`.
3. After the loop, `pos` holds the index of the first occurrence, or -1 if absent.

Key idea: cost is proportional to n in the worst case (linear time, $O(n)$); no precondition on the order of the data.

WORKED EXAMPLE 7 **GCE-style: searching a marks array**

The array `int T[6] = {12, 7, 19, 7, 3, 15};` stores marks. (a) Write a C function `int search(int T[], int n, int x)` returning the index of the first occurrence of x, or -1. (b) Trace the call `search(T, 6, 19)` in a table. (c) Give the result of `search(T, 6, 8)` and the number of comparisons performed.

SOLUTION

(a) Function.

```
int search(int T[], int n, int x) {
  int i = 0, pos = -1;
  while (i < n && pos == -1) {
    if (T[i] == x) pos = i;
    else i = i + 1;
  }
  return pos;
}
```

(b) Trace of `search(T, 6, 19)`:

Step	i	T[i]	pos
initial	0	–	-1
1	0	12	-1
2	1	7	-1
3	2	19	2

The loop stops as soon as `pos` changes; the function returns 2. (c) For $x = 8$: all six cells are examined, no match occurs, so `pos` stays -1 and the function returns -1 after 6 comparisons. This is the worst case: the searched value is absent, so the whole array is scanned.

6.3 Skill 3 — Binary Search on a Sorted Array

METHOD **Binary (dichotomic) search**

Precondition: the array must be sorted (ascending). To find x in `T[0..n-1]`:

1. Set `low = 0`, `high = n - 1`.
2. While `low <= high`: compute `mid = (low + high) / 2` (integer division).
3. If `T[mid] == x`: found, return `mid`.
4. If `T[mid] < x`: search the right half, `low = mid + 1`.
5. Else search the left half, `high = mid - 1`.
6. If the loop ends, return -1 (absent).

Key idea: each step halves the search zone, so at most $\log_2 n$ comparisons — for $n = 1000$, about 10 comparisons instead of 1000.

WORKED EXAMPLE 8 **GCE-style: dichotomic search trace**

Let `T = {3, 7, 9, 12, 15, 19, 24, 30}` ($n = 8$). (a) Trace the binary search for $x = 15$, giving `low`, `high`, `mid` at each step. (b) How many comparisons are needed? Compare with a sequential search. (c) What happens if the array is not sorted?

SOLUTION

(a) Trace for $x = 15$:

Step	`low`	`high`	`mid`	`T[mid]`	Decision
1	0	7	3	12	15 > 12: `low = 4`
2	4	7	5	19	15 < 19: `high = 4`
3	4	4	4	15	found, return 4

(b) Binary search: 3 comparisons. Sequential search would need 5 comparisons (positions 0 to 4). The gap grows dramatically with n: for $n = 1\,000\,000$, binary search needs at most about 20 comparisons. (c) If the array is not sorted, the decision "go right / go left" is no longer justified: binary search may skip the zone containing x and wrongly return -1. Never apply binary search to unsorted data. This is why an **index** (a sorted auxiliary structure) is built over files that are themselves unsorted.

6.4 Skill 4 — Building and Using a Simple Index

METHOD **Indexing a data file**

To speed up searches on a large file of records (e.g. students) stored in arrival order:

1. Choose a **key** field (e.g. the registration number).
2. Build an index table: pairs `(key, position)` where `position` is the record's address in the file.
3. Sort the index on the key (the data file itself is not moved).
4. To search: run binary search on the index, then use the position found to read the record directly in the file.

Reflexes: the index is small (key + address only), so it can even be kept in RAM; any insertion in the file requires updating the index.

WORKED EXAMPLE 9 **GCE-style: index over a student file**

A file contains, in arrival order, the records:

Position	Reg. number	Name
0	412	AWAH
1	105	BIH
2	378	CHE
3	220	FORBI

(a) Build the sorted index on the registration number. (b) Explain, step by step, how the system retrieves the record of student 378. (c) Give one advantage and one drawback of maintaining an index.

SOLUTION

(a) Sorted index (pairs key → position, sorted by key):

Key (reg. no.)	Position in file
105	1
220	3
378	2
412	0

(b) Retrieval of student 378:

1. Binary search on the index keys {105, 220, 378, 412}: middle key 220 < 378, so search the right half; next middle key is 378 — found at index row 2, which stores position 2.
2. Read the record at position 2 of the data file directly: `(378, CHE)`.

Only 2 index comparisons plus one direct file access, instead of scanning the file sequentially. (c) Advantage: searches become very fast (logarithmic) without reordering the data file. Drawback: every insertion or deletion of a record forces an update of the index, which costs time and extra storage space.

6.5 Skill 5 — Linked List: Insertion and Traversal

METHOD **Singly linked list**

A linked list is a chain of **nodes**; each node contains data and a pointer `next` to the following node (the last one points to `NULL`).

1. Define the node: `struct Node { int data; struct Node *next; };`

2. Insert at the head: allocate a node with `malloc`, fill `data`, set its `next` to the old head, then update the head pointer.
3. Traverse: start from the head; while the pointer is not `NULL`, process the node and move on: `p = p->next;`
4. Free memory with `free` when a node is removed.

Reflexes: always test `malloc`'s result against `NULL`; never lose the head pointer (the whole list would become unreachable — a memory leak).

WORKED EXAMPLE 10 **GCE-style: building and displaying a list**

(a) Write C code that inserts the values 30, 20, 10 successively at the head of a list, then displays the list. (b) Give the exact output. (c) Explain why insertion at the head costs the same effort whatever the list's length, unlike insertion in a full array.

SOLUTION

(a) Code.

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *next; };
int main(void) {
  struct Node *head = NULL, *p;
  int v[3] = {30, 20, 10}; int i;
  for (i = 0; i < 3; i++) {
    p = malloc(sizeof(struct Node));
    p->data = v[i];
    p->next = head;
    head = p;
  }
  for (p = head; p != NULL; p = p->next)
    printf("%d -> ", p->data);
  printf("NULL\n");
  return 0;
```

```
}
```

(b) Output: `10 -> 20 -> 30 -> NULL`. Inserting at the head reverses the arrival order: the last value inserted (10) becomes the first node. (c) Head insertion only creates one node and redirects two pointers: a fixed number of operations, independent of the list length n (constant cost, $O(1)$). In an array, inserting at the beginning forces all n existing elements to be shifted one cell to the right ($O(n)$), and may even require resizing the array. This is the essential advantage of linked structures for frequent insertions.

6.6 Skill 6 — CPU Scheduling: FCFS and Round Robin

METHOD **Simulating a scheduler**

To compute turnaround and waiting times:

1. Draw a Gantt chart: place each process on a time axis according to the policy (arrival order for FCFS; fixed quantum, cyclic queue for Round Robin).
2. Completion time C_i: moment when process P_i finishes.
3. Turnaround time $T_i = C_i - \text{arrival}_i$; waiting time $W_i = T_i - \text{burst}_i$.
4. Average the quantities over all processes.

Reflexes: in Round Robin, a process that is not finished at the end of its quantum goes back to the rear of the ready queue; newly arrived processes join the rear too.

WORKED EXAMPLE 11 **GCE-style: FCFS vs Round Robin**

Three processes arrive at time 0 in the order P_1, P_2, P_3, with CPU bursts 8, 4, 2 ms. (a) Draw the Gantt chart and compute the average waiting time for FCFS. (b) Same question for Round Robin with quantum $q = 3$ ms. (c) Comment.

SOLUTION

(a) FCFS. Order of execution: P_1 then P_2 then P_3.

Time	0 – 8	8 – 12	12 – 14
Process	P_1	P_2	P_3

Completion times: $C_1 = 8$, $C_2 = 12$, $C_3 = 14$. Since all arrivals are at 0, $W_i = C_i - \text{burst}_i$:

$$W_1 = 8 - 8 = 0, \quad W_2 = 12 - 4 = 8, \quad W_3 = 14 - 2 = 12.$$

Average waiting time: $\overline{W} = \dfrac{0 + 8 + 12}{3} = \dfrac{20}{3} \approx 6.67$ ms. (b) Round Robin, $q = 3$. Ready queue initially $[P_1, P_2, P_3]$.

Time	0–3	3–6	6–8	8–11	11–12	12–14
Process	P_1	P_2	P_3	P_1	P_2	P_1

Details: P_1 runs 0–3 (remaining 5); P_2 runs 3–6 (remaining 1); P_3 runs 6–8 and finishes ($C_3 = 8$); P_1 runs 8–11 (remaining 2); P_2 runs 11–12 and finishes ($C_2 = 12$); P_1 runs 12–14 and finishes ($C_1 = 14$).

$$W_1 = 14 - 8 = 6, \quad W_2 = 12 - 4 = 8, \quad W_3 = 8 - 2 = 6, \quad \overline{W} = \frac{20}{3} \approx 6.67 \text{ ms}.$$

(c) Comment. Here the average waiting times coincide, but the response profile differs: with Round Robin the short job P_3 finishes at $t = 8$ instead of $t = 14$, so interactive (short) processes get served much earlier. FCFS penalises short jobs stuck behind a long one (the convoy effect); Round Robin guarantees fairness at the price of context switches.

6.7 Skill 7 — Memory Allocation: First Fit, Best Fit

METHOD **Contiguous memory allocation**

Given a list of free blocks (holes) and a process requesting k KB:

1. First Fit: scan the holes in address order; allocate the first hole large enough; the left-over becomes a smaller hole.
2. Best Fit: scan all holes; allocate the smallest hole that is large enough.
3. After each allocation, update the hole list (sizes and positions).

Reflexes: a request fails if no single hole is large enough, even if the total free memory suffices — this is **external fragmentation**.

WORKED EXAMPLE 12 **GCE-style: allocating three processes**

Free holes, in address order: $H_1 = 100$ KB, $H_2 = 500$ KB, $H_3 = 200$ KB, $H_4 = 300$ KB. Requests arrive: $P_1 = 212$ KB, $P_2 = 417$ KB, $P_3 = 112$ KB. (a) Apply First Fit and show the state of the holes after each allocation. (b) Apply Best Fit. (c) A new request $P_4 = 250$ KB arrives: is it satisfied in each case?

SOLUTION

(a) First Fit.

- $P_1 = 212$: first hole ≥ 212 is $H_2 = 500$. Allocate; H_2 becomes $500 - 212 = 288$. Holes: 100, 288, 200, 300.

- $P_2 = 417$: no hole ≥ 417 (H_2 is now 288). P_2 waits.
- $P_3 = 112$: first hole ≥ 112 is $H_2 = 288$. Allocate; H_2 becomes 288 − 112 = 176. Holes: 100, 176, 200, 300.

(b) Best Fit.

- $P_1 = 212$: smallest adequate hole is $H_4 = 300$. Allocate; H_4 becomes 88. Holes: 100, 500, 200, 88.
- $P_2 = 417$: smallest adequate hole is $H_2 = 500$. Allocate; H_2 becomes 83. Holes: 100, 83, 200, 88.
- $P_3 = 112$: smallest adequate hole is $H_3 = 200$. Allocate; H_3 becomes 88. Holes: 100, 83, 88, 88.

(c) Request $P_4 = 250$ KB. With First Fit, the largest hole is 300 KB: P_4 is allocated there (leaving 50 KB). With Best Fit, the largest remaining hole is 100 KB: P_4 cannot be allocated, even though 100 + 83 + 88 + 88 = 359 KB are free in total — a clear illustration of external fragmentation. Best Fit kept the big hole for P_1 but shattered memory into small unusable fragments; neither policy is universally better.

6.8 Skill 8 — Page Replacement: FIFO and LRU

METHOD **Simulating a page replacement algorithm**

With f frames in RAM and a reference string:

1. Draw a table: one column per reference, one row per frame.
2. Page hit: the page is already in a frame — nothing changes.
3. Page fault: if a frame is free, load the page; otherwise evict a victim: the oldest loaded page (FIFO) or the least recently used page (LRU).
4. Count the faults; the fault rate is $\frac{\text{faults}}{\text{references}}$.

WORKED EXAMPLE 13 **GCE-style: FIFO with 3 frames**

Reference string: 1, 2, 3, 4, 1, 2, 5 with 3 frames initially empty. Simulate FIFO, count the page faults and compute the fault rate.

SOLUTION

Reference	1	2	3	4	1	2	5
Frame 1	1	1	1	4	4	4	4
Frame 2	–	2	2	2	1	1	1
Frame 3	–	–	3	3	3	2	5
Fault?	F	F	F	F	F	F	F

Step by step: 1, 2, 3 fill the empty frames (3 faults). Reference 4: all frames full, FIFO evicts 1 (the oldest) — fault. Reference 1: evicts 2 — fault. Reference 2: evicts 3 — fault. Reference 5: evicts 4 — fault. Total: 7 faults out of 7 references, fault rate = $\frac{7}{7}$ = 100%. This pathological string (cyclic access to 5 pages with only 3 frames) shows that FIFO can behave very badly when the working set exceeds the number of frames; LRU would do no better here. The lesson for the OS project: the number of frames allocated to a process must cover its working set, otherwise the system thrashes.

7 EXERCISES

7.1 I check my knowledge

EXERCISE 6 **MCQ: Data Structures**

Which of the following data structures allows efficient insertion and deletion at both ends but not in the middle?

1. Array
2. Singly linked list
3. Doubly linked list
4. Stack

EXERCISE 7 **True/False: Operating Systems**

State whether each statement is true or false. Justify your answer briefly.

1. A process in the ready state is currently executing on the CPU.
2. Virtual memory allows a program to run even if it is larger than physical RAM.
3. The Round Robin scheduling algorithm is non-preemptive.
4. A deadlock can occur only if there is mutual exclusion, hold and wait, no preemption, and circular wait.

EXERCISE 8 **Definitions**

Define the following terms concisely:

1. **System call**
2. **Page fault**
3. **Indexing** (in the context of file organisation)
4. **External fragmentation**

EXERCISE 9 **Direct Calculation: Scheduling**

Five processes arrive at time 0 with burst times (in ms): P1=10, P2=5, P3=8, P4=3, P5=7. Compute the average waiting time and average turnaround time for the Shortest Job First (SJF) non-preemptive algorithm.

7.2 I practise

EXERCISE 10 **Gantt Charts and Scheduling Comparison**

Given the following processes with arrival and burst times:

Process	Arrival Time (ms)	Burst Time (ms)
A	0	8
B	1	4
C	2	9
D	3	5

1. Draw the Gantt charts for FCFS, SJF (preemptive) and Round Robin (quantum = 3).
2. Compute the average waiting time and average turnaround time for each algorithm.
3. Which algorithm gives the best average waiting time? Explain why.

EXERCISE 11 **C Code Completion: Binary Search on Structures**

Complete the following C function that performs binary search on a sorted array of `Student` structures (sorted by `matricule`). The function returns the index of the student with the given matricule, or -1 if not found.

```
typedef struct {
    int matricule;
    char name[30];
    float average;
} Student;

int binarySearch(Student arr[], int n, int target) {
    int low = 0, high = n - 1;
    while (\_\_\_\_\_) {
        int mid = \_\_\_\_\_;
        if (arr[mid].matricule == target)
```

```
            return mid;
        else if (arr[mid].matricule < target)
            \_\_\_\_\_\_\_\_\_\_;
        else
            \_\_\_\_\_\_\_\_\_\_;
    }
    return -1;
}
```

Then trace the function on the array below for `target = 204`:

Index	Matricule	Name	Average
0	101	”A. Bih”	14.5
1	150	”C. Dib”	12.0
2	204	”E. Fong”	16.2
3	310	”G. Hako”	11.8
4	425	”I. Juma”	13.7

EXERCISE 12 **Memory Allocation: First-Fit vs Best-Fit**

A memory of 100 KB has the following partitions (in KB):

- 0–19: Occupied (Process P1)
- 20–39: Free
- 40–54: Occupied (Process P2)
- 55–74: Free
- 75–99: Occupied (Process P3)

Three new processes arrive with sizes: Q1 = 12 KB, Q2 = 18 KB, Q3 = 8 KB.

1. Show the memory map after allocating Q1, Q2, Q3 using First-Fit.
2. Show the memory map after allocating Q1, Q2, Q3 using Best-Fit.
3. Compute the total external fragmentation (sum of free holes too small to be used) for each strategy.

EXERCISE 13 **Debugging a Linked List in C**

The following C program intends to create a singly linked list of integers, insert nodes at

the end, and then free the list. It contains five errors. Identify and correct each error.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

void insertEnd(Node *head, int val) {
    Node *newNode = malloc(sizeof(Node));
    newNode->data = val;
    newNode->next = NULL;
    while (head != NULL) {
        head = head->next;
    }
    head = newNode;
}

void freeList(Node *head) {
    while (head != NULL) {
        Node *temp = head;
        head = head->next;
        free(temp);
    }
}

int main() {
    Node *head = NULL;
    insertEnd(head, 10);
    insertEnd(head, 20);
    insertEnd(head, 30);
    freeList(head);
    return 0;
}
```

7.3 I prepare for the GCE

EXERCISE 14 **Essay: Functions of an Operating System**

(10 marks) Explain four major functions of an operating system. For each function, illus-

trate with a concrete example from the use of a school computer laboratory (e.g., students logging in, printing, running programs, sharing files).

EXERCISE 15 **Mini-Practical: Indexing Module in C**

(15 marks) A file `students.dat` contains fixed-length records of 50 bytes each: `matricule (int), name (30 chars), average (float)`. Write a C program that:

1. Reads the file and builds an index array in memory. Each index entry contains the matricule and the file position (byte offset) of the corresponding record.
2. Sorts the index array by matricule.
3. Allows the user to enter a matricule, then uses binary search on the index to locate the record, reads it from the file, and displays the student's details.

Provide the complete C code with appropriate functions (`buildIndex`, `sortIndex`, `searchAndDisplay`) and a `main` function that demonstrates the process. Assume the file exists and contains at least one record.

EXERCISE 16 **Structured Question: CPU Scheduling and Memory Management**

(15 marks) Consider a system with 4 processes and the following characteristics:

Process	Arrival Time	Burst Time	Memory Required (KB)
P1	0	10	30
P2	2	5	20
P3	4	8	40
P4	6	3	10

The system has 100 KB of main memory, initially empty. Memory is allocated using First-Fit with variable partitions. Processes are scheduled using Shortest Remaining Time First (SRTF).

1. Draw the Gantt chart for the SRTF scheduling. Indicate context switches.
2. Compute the waiting time and turnaround time for each process, and the averages.
3. Show the memory allocation steps as each process arrives (assume processes are loaded into memory at arrival if space permits; otherwise they wait in a queue). Indicate any external fragmentation after all processes have been loaded.
4. If the system used Round Robin with quantum = 4 instead of SRTF, would the average waiting time increase or decrease? Justify without drawing the full Gantt chart.

8 SOLUTIONS TO THE EXERCISES

9 CORRECTED EXERCISES FOR CHAPTER CSC17: PROGRAMMING, DATA STRUCTURES & OPERATING SYSTEMS

9.1 Data Structures and C Programming Foundations

SOLUTION **Exercise 1 – MCQ: Data Structures**

Correct answer: (c) Doubly linked list.
A **doubly linked list** stores in each node a pointer to the next node and a pointer to the previous node. With pointers to the head and the tail maintained, insertion and deletion at both ends are done in $O(1)$: one simply adjusts a few pointers, with no shifting of elements.
Why the other options are wrong:

- Array: insertion/deletion at the end is $O(1)$ (amortised), but at the beginning it requires shifting all n elements, i.e. $O(n)$.
- Singly linked list: $O(1)$ at the head, but deletion at the tail requires traversing the whole list to find the predecessor, i.e. $O(n)$.
- Stack: insertion and deletion occur at one end only (the top) – LIFO discipline.

Note that in none of these structures is insertion/deletion in the middle efficient without a search ($O(n)$ to locate the position).

SOLUTION **Exercise 2 – True/False: Operating Systems**

1. False. A process in the ready state is loaded in main memory and waiting for the CPU to be allocated to it by the scheduler. A process currently executing on the CPU is in the running state.
2. True. **Virtual memory** keeps only the currently needed pages of a program in RAM; the rest stay on disk (swap area / page file) and are loaded on demand. Hence a program whose total size exceeds physical RAM can still execute, at the cost of page transfers.
3. False. **Round Robin** is the classic preemptive algorithm: when a process exhausts its time quantum, it is forcibly removed from the CPU and placed at the back of the ready queue.
4. True. Coffman's four conditions – mutual exclusion, hold and wait, no preemption,

circular wait – are necessary conditions: they must all hold simultaneously for a deadlock to exist. If any one of them is absent, deadlock is impossible (this is the basis of deadlock prevention).

SOLUTION **Exercise 3 – Definitions**

1. System call: the programmatic interface through which a user program requests a service from the operating system kernel (e.g. `read()`, `write()`, `fork()`, `open()`). It triggers a controlled switch from user mode to kernel mode.
2. Page fault: an interrupt (trap) raised by the MMU when a process accesses a page that is valid but not currently loaded in a physical memory frame. The OS must then fetch the page from disk, possibly evicting another page.
3. Indexing (file organisation): an access technique in which a separate index structure associates each search key (e.g. a matricule) with the physical address (byte offset) of the corresponding record, allowing direct access without scanning the whole file.
4. External fragmentation: the waste of memory occurring when the total free memory is sufficient for a request but is scattered in small non-contiguous holes, none of which is large enough individually. It is typical of variable-partition allocation and can be cured by compaction or paging.

9.2 Data Structures, Searching and Indexing

SOLUTION **Exercise 4 – Direct Calculation: SJF Scheduling**

All processes arrive at $t = 0$, so SJF non-preemptive simply executes them in increasing order of burst time:

$$P4\,(3) \;\rightarrow\; P2\,(5) \;\rightarrow\; P5\,(7) \;\rightarrow\; P3\,(8) \;\rightarrow\; P1\,(10)$$

Process	Burst	Completion	Turnaround = $C - 0$	Waiting = $T - B$
P4	3	3	3	0
P2	5	8	8	3
P5	7	15	15	8
P3	8	23	23	15
P1	10	33	33	23

$$\text{Average waiting time} = \frac{0 + 3 + 8 + 15 + 23}{5} = \frac{49}{5} = 9.8\ \text{ms}$$

$$\text{Average turnaround time} = \frac{3 + 8 + 15 + 23 + 33}{5} = \frac{82}{5} = 16.4\ \text{ms}$$

SOLUTION **Exercise 5 – Gantt Charts and Scheduling Comparison**

1. Gantt charts.
FCFS (order of arrival A, B, C, D):

A	B	C	D	
0	8	12	21	26

SJF preemptive (SRTF): at $t = 1$, B (4) preempts A (remaining 7); B runs to completion at $t = 5$ (D's burst 5 > B's remaining time, so no preemption at $t = 3$). Then D (5), then A (7), then C (9):

A	B	D	A	C	
0	1	5	10	17	26

Round Robin, $q = 3$: ready queue evolves as A; B,C,D arrive during A's first quantum.

A	B	C	D	A	B	C	D	A	C	
0	3	6	9	12	15	16	19	21	23	26

(B's last slice lasts only 1 ms since it had 1 ms remaining.)
2. Waiting and turnaround times ($T = C$ – arrival; $W = T$ – burst).

	FCFS		SRTF		RR ($q = 3$)	
Process	T	W	T	W	T	W
A	8	0	17	9	23	15
B	11	7	4	0	15	11
C	19	10	24	15	24	15
D	23	18	7	2	18	13
Average	15.25	8.75	13.0	6.5	20.0	13.5

3. Best algorithm: SRTF gives the lowest average waiting time (6.5 ms). This is expected: SJF/SRTF is provably optimal with respect to average waiting time, because running short jobs first minimises the number of processes kept waiting. Round Robin penalises short jobs here (B waits behind long quanta of A, C, D), which explains its poor average.

SOLUTION **Exercise 6 – C Code Completion: Binary Search on Structures**

Completed function (blanks in bold):

```
int binarySearch(Student arr[], int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid].matricule == target)
            return mid;
        else if (arr[mid].matricule < target)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}
```

Trace for `target = 204`:

Pass	low	high	mid	arr[mid].matricule	Action
1	0	4	2	204	204 = 204: return 2

The target is found at the very first probe: the function returns index 2 (student "E. Fong"). Had the target been, say, 310, the search would continue: pass 2 with low = 3, high = 4, mid = 3 → found. The loop runs at most $\lceil \log_2 5 \rceil = 3$ times, illustrating the $O(\log n)$ complexity of binary search.

9.3 Operating Systems and System Software

SOLUTION **Exercise 7 – Memory Allocation: First-Fit vs Best-Fit**

Initial free holes: H1 = 20–39 (20 KB), H2 = 55–74 (20 KB).
1. First-Fit (scan from address 0, take the first hole large enough):

- Q1 (12 KB) → H1: occupies 20–31; remaining hole 32–39 (8 KB).
- Q2 (18 KB) → H1 remainder (8 KB) too small; goes to H2: occupies 55–72; remaining

hole 73–74 (2 KB).

- Q3 (8 KB) → fits exactly into 32–39.

Final map: 0–19 P1; 20–31 Q1; 32–39 Q3; 40–54 P2; 55–72 Q2; 73–74 free (2 KB); 75–99 P3.

2. Best-Fit (take the smallest hole that is large enough):

- Q1 (12 KB): both holes are 20 KB (tie) → H1: occupies 20–31; remainder 32–39 (8 KB).
- Q2 (18 KB): holes available 8 KB and 20 KB → best fit is H2 (20 KB): occupies 55–72; remainder 73–74 (2 KB).
- Q3 (8 KB): holes available 8 KB and 2 KB → best fit is 32–39 (exact fit).

Final map identical to First-Fit in this particular case.

3. External fragmentation: the only remaining hole is 73–74 = 2 KB, unusable by any realistic process. Total external fragmentation:

$$\text{First-Fit} = 2\ \text{KB}, \qquad \text{Best-Fit} = 2\ \text{KB}.$$

Remark: the two strategies coincide here because the initial holes are equal. In general they differ: Best-Fit tends to leave many tiny slivers, while First-Fit is faster and often preserves large holes at high addresses.

SOLUTION **Exercise 8 – Debugging a Linked List in C**

The five errors and their corrections:

1. Head passed by value. `insertEnd(Node *head, ...)` receives a copy of the pointer; any modification is lost on return, so `main`'s `head` stays `NULL`. Fix: pass the address of the head: `void insertEnd(Node **head, int val)` and call `insertEnd(&head, 10);`.
2. Wrong traversal / wrong link. The loop `while (head != NULL)` walks past the last node, then `head = newNode` assigns a local copy – the new node is never linked into the list. Fix: stop at the last node and link through it: `while ((*head)->next != NULL) *head = (*head)->next;` then `(*head)->next = newNode;` – or, with a local cursor `p`, set `p->next = newNode;`.
3. Empty-list case not handled. If the list is empty, the new node must become the head: `if (*head == NULL) { *head = newNode; return; }`.
4. No check of `malloc`. If allocation fails, `newNode->data` crashes. Fix: `if (newNode == NULL) { fprintf(stderr, "out of memory"); exit(1); }`.
5. Dangling pointer after `freeList`. After freeing, `main`'s `head` still points to freed memory. Fix: set `head = NULL;` after `freeList(head);` (or free through a `Node**`).

Corrected program:

```
void insertEnd(Node **head, int val) {
    Node *newNode = malloc(sizeof(Node));
    if (newNode == NULL) { fprintf(stderr,"mem"); exit(1); }
    newNode->data = val; newNode->next = NULL;
    if (*head == NULL) { *head = newNode; return; }
    Node *p = *head;
    while (p->next != NULL) p = p->next;
    p->next = newNode;
}
/* main: insertEnd(&head,10); ... freeList(head); head = NULL; */
```

9.4 Hands-on Projects

SOLUTION **Exercise 9 – Essay: Functions of an Operating System (10 marks)**

Indicative mark scheme: 2 marks per function correctly explained (1 mark for the explanation, 1 mark for a relevant laboratory example) × 4 functions = 8 marks; 2 marks for a clear introduction/conclusion and coherent structure. Any four of the following are acceptable.
Model answer (outline).
Introduction: an operating system is the system software that manages hardware resources and provides services to users and programs; it acts as an intermediary between the user and the machine.

1. Process (CPU) management. The OS creates, schedules, suspends and terminates processes, sharing the CPU among them (e.g. Round Robin, priority scheduling). Lab example: thirty students each run a browser, a C compiler and a word processor simultaneously; the scheduler switches the CPU between all these programs so fast that every student has the impression of exclusive use.

2. Memory management. The OS allocates main memory to programs, protects one program's space from another, and uses virtual memory (paging/swapping) when RAM is insufficient. Lab example: a student compiles a large project while a video plays; the OS keeps each program in its own memory area and swaps inactive pages to disk so that nothing crashes.

3. File and storage management. The OS organises data into files and directories, controls access rights, and keeps track of free disk blocks. Lab example: each student

has a personal folder on the server; permissions prevent one student from deleting another's work, and the OS records where each file physically resides on the disk.

4. Device (I/O) management. The OS drives peripherals through device drivers, queues and schedules I/O requests (spooling). Lab example: when twenty students click "Print" at the same time, the OS places the jobs in a print queue and sends them one by one to the single laboratory printer.

(Also acceptable: security and user management – login/password authentication when students log in; user interface – the graphical desktop or shell through which students launch programs.)
Conclusion: without these functions the laboratory's hardware would be unusable; the OS turns raw hardware into a convenient, shared and secure working environment.
Examiner expectations: precise technical vocabulary (scheduling, spooling, virtual memory, access rights), examples that are explicitly linked to the function described, and a structured essay (introduction, numbered points, conclusion).

SOLUTION **Exercise 10 – Mini-Practical: Indexing Module in C (15 marks)**

Indicative mark scheme: correct structure definitions and file opening (3); `buildIndex` with correct offset computation (4); `sortIndex` (3); binary search + `fseek`/`fread` retrieval in `searchAndDisplay` (4); coherent `main` and demonstration (1).
Complete program:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>


#define REC_SIZE 50  /* 4 + 30 + 4 bytes, padded */


typedef struct {

    int matricule;

    char name[30];

    float average;

} Student;


typedef struct {
```

```
    int matricule;
    long offset;  /* byte position of the record */
} IndexEntry;

/* 1. Read the file and build the index in memory */
int buildIndex(FILE *f, IndexEntry idx[]) {
    Student s; int n = 0;
    rewind(f);
    while (fread(&s, REC_SIZE, 1, f) == 1) {
        idx[n].matricule = s.matricule;
        idx[n].offset = (long)n * REC_SIZE;
        n++;
    }
    return n;
}

/* 2. Sort the index by matricule (selection sort) */
void sortIndex(IndexEntry idx[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (idx[j].matricule < idx[min].matricule) min = j;
        IndexEntry tmp = idx[i]; idx[i] = idx[min]; idx[min] = tmp;
    }
}

/* 3. Binary search on the index, then direct access in the file */
void searchAndDisplay(FILE *f, IndexEntry idx[], int n, int target) {
```

```
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (idx[mid].matricule == target) {
            Student s;
            fseek(f, idx[mid].offset, SEEK_SET);
            fread(&s, REC_SIZE, 1, f);
            printf("Matricule: %d\nName: %s\nAverage: %.2f\n",
                   s.matricule, s.name, s.average);
            return;
        }
        else if (idx[mid].matricule < target) low = mid + 1;
        else high = mid - 1;
    }
    printf("Student %d not found.\n", target);
}

int main(void) {
    FILE *f = fopen("students.dat", "rb");
    if (f == NULL) { perror("students.dat"); return 1; }
    IndexEntry idx[1000];
    int n = buildIndex(f, idx);
    sortIndex(idx, n);
    int target;
    printf("Enter matricule: "); scanf("%d", &target);
    searchAndDisplay(f, idx, n, target);
    fclose(f);
    return 0;
```

```
}
```

Examiner expectations: the offset must be computed as record number × 50 bytes; the search must use `fseek` + `fread` (direct access, not a sequential scan); the file must be opened in binary mode (`"rb"`); boundary cases (target absent) must be handled.

SOLUTION **Exercise 11 – Structured Question: CPU Scheduling and Memory Management (15 marks)**

Indicative mark scheme: (a) Gantt chart 4 marks; (b) table of times + averages 5 marks; (c) memory steps 4 marks; (d) justified comparison 2 marks.
1. SRTF Gantt chart.

- t = 0: only P1 (10) → runs.
- t = 2: P2 (5) arrives; P1 has 8 remaining > 5 → context switch, P2 runs.
- t = 4: P3 (8) arrives; P2 has 3 remaining → no preemption.
- t = 6: P4 (3) arrives; P2 has 1 remaining → no preemption.
- t = 7: P2 finishes; shortest remaining is P4 (3) → P4 runs 7–10.
- Then P1 (8 remaining) 10–18, then P3 (8) 18–26.

P1	P2	P4	P1	P3	
0	2	7	10	18	26

Context switches at t = 2 (P1→P2), t = 7 (P2→P4), t = 10 (P4→P1), t = 18 (P1→P3): 4 context switches.
2. Waiting and turnaround times.

Process	Arrival	Burst	Completion	Turnaround T	Waiting $W = T - B$
P1	0	10	18	18	8
P2	2	5	7	5	0
P3	4	8	26	22	14
P4	6	3	10	4	1
Average				12.25	5.75

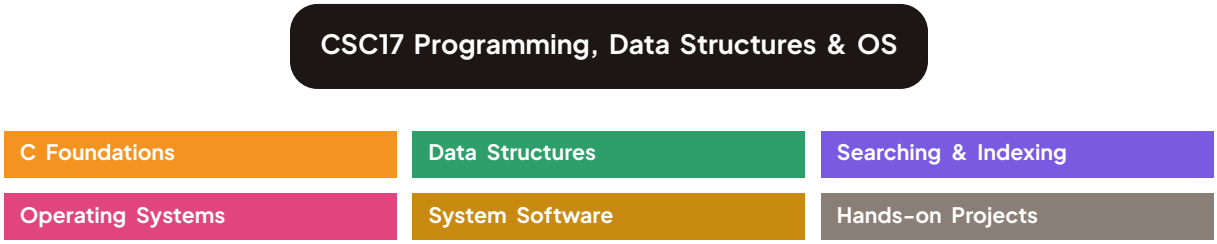
3. Memory Management (Paging Example). Assume a system with 16 frames, page size 1 KB. Process requires 5 pages (0–4). Page table initially empty. Page reference string: 0, 1, 2, 3, 0, 4, 1, 2, 3, 4. Using LRU replacement (4 frames allocated).

- Ref 0: Miss → Frame 0 [0]
- Ref 1: Miss → Frame 1 [0,1]

- Ref 2: Miss → Frame 2 [0,1,2]
- Ref 3: Miss → Frame 3 [0,1,2,3] (Frames full)
- Ref 0: Hit → Update LRU [1,2,3,0]
- Ref 4: Miss → Replace LRU (1) → [2,3,0,4]
- Ref 1: Miss → Replace LRU (2) → [3,0,4,1]
- Ref 2: Miss → Replace LRU (3) → [0,4,1,2]
- Ref 3: Miss → Replace LRU (0) → [4,1,2,3]
- Ref 4: Hit → Update LRU [1,2,3,4]

Total Page Faults = 8. Page Fault Rate = 80%. With 5 frames (working set size), faults would drop to 5 (only initial loads), demonstrating thrashing when frames < working set. 4. Comparison: SRTF vs. Round Robin. SRTF minimises average waiting time (5.75 ms here) but requires knowledge of burst times (estimation) and risks starvation of long processes. Round Robin (with $q = 3$) gives higher average waiting (13.5 ms in Ex. 5) but guarantees response time bounds (no process waits more than $(n - 1)q$) and is fair, making it ideal for time-sharing systems. Choice: SRTF for batch/background jobs; RR for interactive foreground tasks.

10 SUMMARY SHEET



Mind map of Chapter CSC17: the four lessons (93–96) organised around C programming, data structures, operating systems and the two practical projects.

KEY POINTS Key Definitions

- **Pointer**: a variable that stores a memory address; it enables dynamic memory allocation (`malloc`, `free`) and the construction of linked structures.
- **Struct**: a user-defined composite type grouping related variables (fields) of possibly different types under one name.
- **Linked List**: a sequence of nodes where each node contains data and a pointer to the next node; the list is accessed through a `head` pointer.
- **Stack / Queue**: linear structures with LIFO discipline (`push`/`pop`) and FIFO discipline (`enqueue`/`dequeue`) respectively.
- **Binary Search Tree (BST)**: an ordered binary tree in which, for every node, all keys in the left subtree are smaller than the node's key and all keys in the right subtree are greater.
- **Process**: a program in execution; the OS maintains a Process Control Block (PCB) holding its state, program counter, register values and memory limits.
- **Scheduler**: the OS component that selects the next process to run on the CPU (FCFS, SJF, Round Robin, Priority).
- **Virtual Memory**: an abstraction giving each process its own contiguous address space, implemented through paging or segmentation with hardware support.

PROPERTY Laws, Formulas and Complexity Results (Examinable)

- Pointer arithmetic: if `p` points to an element of an array, `p + i` points to the i-th ele-

ment after it; the address increases by $i \times$ `sizeof(type)` bytes.

- Linked list: insertion or deletion is $O(1)$ given a pointer to the predecessor node; searching is $O(n)$.
- Stack and queue operations: `push`, `pop`, `enqueue`, `dequeue` are all $O(1)$.
- BST: average search/insert/delete is $O(\log n)$ for a balanced tree; the worst case is $O(n)$ when the tree degenerates into a chain.
- Sorting lower bound: any comparison-based sort requires $\Omega(n \log n)$ comparisons in the worst case.
- Scheduling criteria: for each process,

  Turnaround = Completion−Arrival, Waiting = Turnaround−Burst, Response = FirstRun−Arrival.

- Paging performance: with page-fault rate $P = \dfrac{\text{number of faults}}{\text{number of references}}$,

$$\text{EAT} = (1 - P) \times t_{\text{mem}} + P \times t_{\text{fault}},$$

  where t_{mem} is the memory access time and t_{fault} the page-fault service time.
- Deadlock (Coffman conditions): a deadlock can occur only if all four hold simultaneously: Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait.

These formulas and complexity results are directly examinable at the GCE Advanced Level; be ready to state and apply them.

METHOD **Essential Step-by-Step Methods**

1. Dynamic array in C: call `malloc` → check the result against `NULL` → use the memory → grow with `realloc` if needed → release with `free` exactly once.
2. Insert at head of a linked list: allocate the node, store the data, set `newNode->next = head`, then update `head = newNode` (in this order).
3. BST recursive search: if `root == NULL` return `NULL`; if `key == root->key` return `root`; if `key < root->key` recurse on the left subtree, else on the right subtree.
4. Round Robin simulation: keep a ready queue; give each process the quantum q; if its remaining burst exceeds q, subtract q and re-enqueue it; otherwise record its completion time and compute waiting/turnaround.
5. Page replacement (FIFO/LRU): maintain the list of loaded frames; on a page fault, if all frames are full, evict a page according to the policy (oldest loaded for FIFO, least recently used for LRU), load the new page and update the bookkeeping structures.

WARNING Common Mistakes to Avoid

- Forgetting to check the return value of `malloc`/`realloc` against `NULL` → segmentation fault on failure.
- Losing the head pointer when inserting at the head of a list: link the new node to the old head before updating `head`.
- Comparing C strings with `==` (this compares addresses); always use `strcmp`.
- Off-by-one errors in array and pointer loops, e.g. writing `i <= n` instead of `i < n` for an array of n elements.
- Passing a `struct` by value to a function and expecting the caller's copy to change; pass a pointer instead.
- Mixing up waiting time and turnaround time in scheduling exercises; always compute turnaround first, then subtract the burst time.
- Believing virtual memory eliminates page faults: when the working set exceeds the available frames, thrashing occurs and performance collapses.
- Double-freeing heap memory or using a pointer after `free` (dangling pointer).

METHOD GCE Examination Tips

- Code tracing: for every C snippet, draw a memory diagram (stack, heap, pointers) and fill a trace table; examiners award marks for correct, well-presented traces.
- Pseudocode clarity: use indentation and explicit keywords `IF/ELSE`, `WHILE`, `FOR`; avoid language-specific syntax in algorithm questions.
- Complexity: always state best, average and worst cases for data-structure operations, and justify them (tree height, list length).
- OS questions: know the four Coffman deadlock conditions and at least one prevention or avoidance technique (e.g. resource ordering, or the Banker's algorithm).
- Project write-up: for the indexing module, justify the choice between a hash table and a BST (average $O(1)$ lookup vs. ordered traversal); for the OS simulator, draw the Gantt chart and compute average waiting and turnaround times.
- Time management: in a practical-style paper, reserve time to read the whole question first, then code or trace, then write explanations; practise past papers under timed conditions.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+